

ОГЛАВЛЕНИЕ

1.	СПИСОК СОКРАЩЕНИЙ	3
2.	ВВЕДЕНИЕ	5
3.	ЦЕЛИ, ЗАДАЧИ, ПРОБЛЕМАТИКА ПРОЕКТА	6
3.1	ЦЕЛИ ПРОЕКТА	6
3.2	ЗАДАЧИ ПЕРВОГО ЭТАПА ПРОЕКТА	8
3.3	ЗАДАЧИ ВТОРОГО ЭТАПА ПРОЕКТА	9
3.4	АРХИТЕКТУРНЫЕ РЕШЕНИЯ УРОВНЯ ПЛК	11
3.5	АРХИТЕКТУРНЫЕ РЕШЕНИЯ УРОВНЯ HUB	13
3.6	ОПРЕДЕЛЕНИЕ ОСНОВНЫХ РИСКОВ ПРОЕКТА	14
4.	ЭТАП 1. РАЗРАБОТКА ФРЕЙМВОРКА "HORIZON AUTOMATION PLC"	17
4.1	РАЗРАБОТКА МОДУЛЯ NETWORK	17
4.2	РАЗРАБОТКА МОДУЛЯ WSSERVER	18
4.3	РАЗРАБОТКА МОДУЛЯ PROXYWS	19
4.4	РАЗРАБОТКА МОДУЛЯ ROUTEREPL	20
4.5	РАЗРАБОТКА МОДУЛЯ MQTTGW	21
4.6	РАЗРАБОТКА МОДУЛЯ PROXYMQTT	22
4.7	РАЗРАБОТКА МОДУЛЯ LHP	23
4.8	РАЗРАБОТКА МОДУЛЯ PROCESS	25
4.9	РАЗРАБОТКА МОДУЛЯ DEVICEMANAGER	27
5.	ЭТАП 2. РАЗРАБОТКА ФРЕЙМВОРКА "HORIZON AUTOMATION SERVER"	28
5.1	РАЗРАБОТКА МОДУЛЯ PROCESS	28
5.2	РАЗРАБОТКА МОДУЛЯ PROVIDERMDB	29
5.3	РАЗРАБОТКА МОДУЛЯ LOGGER	31
5.4	РАЗРАБОТКА МОДУЛЯ PROXYLOGGER	33
5.5	РАЗРАБОТКА МОДУЛЯ DEVICEMANAGER	35
5.6	РАЗРАБОТКА МОДУЛЯ CHANNEL	36
5.7	РАЗРАБОТКА МОДУЛЯ PROXYCHANNEL	39
5.8	РАЗРАБОТКА МОДУЛЯ WSCLIENT	40
5.9	РАЗРАБОТКА МОДУЛЯ PROXYWSCLIENT	41
5.10	РАЗРАБОТКА МОДУЛЯ MQTTCLIENT	42
5.11	РАЗРАБОТКА МОДУЛЯ PROXYMQTTCLIENT	43
5.12	РАЗРАБОТКА МОДУЛЯ MQTTGW	44
6.	ЗАКЛЮЧЕНИЕ	45
7.	КРАТКИЕ ВЫВОДЫ ПО РЕЗУЛЬТАТАМ ВЫПОЛНЕНИЯ НИОКР	50
7.1	РАЗРАБОТАНА АРХИТЕКТУРА ПТК "HORIZON AUTOMATION"	50

7.2	КЛЮЧЕВЫЕ ДОСТИЖЕНИЯ	50
7.3	ТЕСТИРОВАНИЕ И ВАЛИДАЦИЯ	50
7.4	ПРАКТИЧЕСКАЯ ЗНАЧИМОСТЬ.....	51
7.5	ДАЛЬНЕЙШИЕ НАПРАВЛЕНИЯ	51
8.	ОЦЕНКА ПОЛНОТЫ ВЫПОЛНЕНИЯ ПОСТАВЛЕННЫХ ЗАДАЧ.....	52
9.	РЕКОМЕНДАЦИИ И ИСХОДНЫЕ ДАННЫЕ ДЛЯ ИСПОЛЬЗОВАНИЯ РЕЗУЛЬТАТОВ НИОКР	54
10.	ОЦЕНКА ЭКОНОМИЧЕСКОЙ ЭФФЕКТИВНОСТИ	58
11.	ОЦЕНКА НАУЧНО-ТЕХНИЧЕСКОГО УРОВНЯ ВЫПОЛНЕННОЙ НИОК В СРАВНЕНИИ С ЛУЧШИМИ ДОСТИЖЕНИЯМИ.....	60

1. СПИСОК СОКРАЩЕНИЙ

ПО	- Программное обеспечение
БД	- База данных
СУБД	- Система Управления Базами Данных
Espruino	- Интерпретатор JavaScript с открытым исходным кодом, созданный для микроконтроллеров, в частности для устройств с малым размером оперативной памяти
JS	- JavaScript мультипарадигменный язык программирования, является реализацией спецификации ECMAScript
WI-FI	- Технология беспроводной локальной сети с устройствами на основе стандартов IEEE 802.11
AC	- Access point Точка доступа с Wi-fi
I2C	- Inter Integrated Circuit Последовательная асимметричная шина для связи между интегральными схемами
SPI	- Serial Peripheral Interface Последовательный периферийный интерфейс, синхронная шина для связи между интегральными схемами
UART	- Universal Asynchronous Receiver-Transmitter Универсальный асинхронный приёмопередатчик
MQTT	- Message Queuing Telemetry Transport упрощённый сетевой протокол, работающий поверх TCP/IP, ориентированный на обмен сообщениями между устройствами по принципу «издатель — подписчик».
WebSocket	- Протокол передачи данных, сокращается до WS
REPL	- Read-Eval-Print Loop „Чтение — Вычисление — Вывод“ форма организации интерактивной среды программирования в рамках средств интерфейса командной строки
SSH	- Secure Shell сетевой протокол прикладного уровня, позволяющий производить удалённое управление операционной системой и туннелирование TCP-соединений
LHP	- Lite Horizen Protocol Легкий протокол «Горизонт», разработанный в рамках проекта. Базируется на основе прикладного протокола WebSocket и JSON
JSON	- JavaScript Object Notation

	Текстовый формат обмена данными
PPM	- Parts per million - миллионные доли Единица измерения концентрации вещества в газе
PPB	- Parts per billion - миллиардные доли Единица измерения концентрации вещества в газе
RH	- Относительная влажность воздуха в процентах
МЭМС	- Микроэлектромеханические системы Устройства, объединяющие в себе взаимосвязанные механические и электрические компоненты
ПЗУ	- Постоянное запоминающее устройство
ОЗУ	- Оперативное запоминающее устройство
PSRAM	- Pseudo Static Random Access Memories Псевдостатического ОЗУ
ПЛК	- Программируемый Логический Контроллер
Hub	- Концентратор Мощный контроллер автоматизации, построенный на базе компактного промышленного микрокомпьютера развитыми коммуникационными возможностями
ПТК	- Программно-Технический Комплекс

2. ВВЕДЕНИЕ

Данный отчет отражает содержание и результаты научно-исследовательских, опытно конструкторских и инженерных работ, проведенных в рамках договора № 5152ГС1/89582 по теме: «Разработка прототипа мобильной роботизированной платформы контроллера» (Проект № 89582, заявка №С1-315208) в соответствии с объемом предусмотренным договором.

В соответствии с намеченным планом проекта должен был быть разработан программно-технический комплекс, который в ходе работ получил наименование – "Horizon Automation".

В ходе реализации проекта были разработаны два фреймворка:

- "Horizon Automation PLC", предназначен для управления ПЛК;
 - "Horizon Automation Server", предназначен для управления Hub.
- По обеим фреймворкам подана заявка на получение патента.

Далее будет подробно рассмотрен ход реализации задуманного функционала, и проблематика, с которой столкнулась команда разработчиков, а также насколько удалось достичь намеченных ТТХ проекта.

3. ЦЕЛИ, ЗАДАЧИ, ПРОБЛЕМАТИКА ПРОЕКТА

3.1 ЦЕЛИ ПРОЕКТА

Цель проекта – разработка программно-технического комплекса, представляющего собой многоуровневую, интегрируемую, многофункциональную платформу автоматизации, состоящую из фреймворков "нижнего" и "верхнего" уровней. Предназначенного для автоматизации широкого круга прикладных задач, а также организации образовательных программ для различного уровня учебных заведений.

Реализация указанных целей требовала решения следующих задач:

- разработка архитектуры ПТК в целом и каждого из фреймворков в отдельности;
- написание и отладка программного обеспечения фреймворков;
- отбор на рынке существующих аппаратных, решений для аппаратной платформы ПТК и Hub;
- адаптация и развертывание на них соответствующие фреймворков, проведение множественных тестов в разной комбинации аппаратного обеспечения. Обеспечение стабильной работы в объеме задуманного функционала.

Как было сказано выше фреймворки "нижнего" и "верхнего" уровней получили соответственно названия:

- "Horizon Automation PLC";
- "Horizon Automation Server".

Первый из них отвечает за работу ПЛК на "нижнем", полевом, уровне автоматизации. Второй отвечает за работу Hub, т.е. за работу ПТК на "верхнем", серверном, уровне автоматизации.

Далее даются краткие комментарии по порядку реализации задач и критически важных этапов.

На первом этапе разрабатывались все стратегические решения в части аспектов архитектуры ПТК.

На следующем этапе реализации проекта требовалось разработать и отладить программный код основных системных и прикладных служб фреймворков "нижнего" и "верхнего" уровней. Это самая сложная и продолжительная фаза проекта.

Сложность разработки этого этапа усугубляется тем, что помимо архитектурных решений, которые формирует команда проекта, необходимо реализовать большой набор поддержки драйверов всевозможных сенсоров и актуаторов которые активно производятся и пользуются спросом на рынке автоматизации. Для этого необходимо провести соответствующий анализ, отобрать базовый набор устройств, закупить, изучить документацию и разработать драйвера.

На серверном уровне есть аналогичная проблема, но там она выливается в определение перечня специфичных источников и внешних служб поддержку которых необходимо разработать. Здесь также предстояло

провести глубокий анализ рынка, определить какие протоколы и источники востребованы, далее также изучить документацию и написать драйвера обеспечивающие их поддержку в платформе ПТК "Horizon Automation Server".

От того насколько удачно команда проекта отберет ограниченный реестр устройств "нижнего" уровня (сенсоры, актуаторы) и служб и протоколов на "верхнем" уровне, во многом зависит успешность вывода ПТК на рынок автоматизации.

Помимо достижения технических и финансовых показателей комплекса, перед командой проекта стоял целый ряд задач, связанных с определением и преодолением рисков проекта. Требовалось на старте проекта определить ключевые риски, нивелировать их, либо найти обходные пути получения требуемого функционала.

3.2 ЗАДАЧИ ПЕРВОГО ЭТАПА ПРОЕКТА

Главной задачей первого этапа проекта – на первом этапе проекта требовалось разработать архитектуру "нижнего" уровня фреймворка "Horizon Automation PLC", разработать и отладить код основных системных и прикладных служб фреймворка, нивелировать выявленные на старте проекта риски либо найти обходные пути получения требуемого функционала.

Фреймворк полевого уровня – "Horizon Automation PLC" предназначен для установки на 32-разрядный программируемый логический контроллер с архитектурой ARM серии Cortex M4 или старше, либо Tensilica Xtensa LX6/LX7. Данные архитектуры принадлежат, разрабатываются, производятся и поддерживаются разными производителями, что делает платформу очень устойчивой к негативным геополитическим факторам.

Исполнительная среда фреймворка – Espruino является open source продуктом. Все взаимодействие и разработка прикладного кода в рамках фреймворка ведется на языке JavaScript, с открытым стандартом.

Понять успешная ли концепция была заложена при разработке проекта можно только одним способом, разработать большое количество модулей задуманного фреймворка "Horizon Automation PLC" и протестировать его на реальных микропроцессорных модулях. Нередко при разработке подобных систем, достаточно проверить работоспособность идеи в некотором ограниченном количестве «реперных» точек и тогда успешное тестирование будет означать успех проекта, в случае неуспеха это будет означать и не реализуемость проекта.

В нашем случае, к сожалению, это не так. Мы поставили перед собой весьма амбициозную задачу – разработать высокоуровневый фреймворк для нишевых микропроцессорных систем с очень скромными аппаратными ресурсами. И при этом претворить в жизнь передовые разработки в области языка JavaScript (поддержка стандарта ECMAScript 6/7). Предоставить пользователю возможности на миниатюрном микроконтроллере которые ранее были доступны только на полноценных компьютерных системах!

Но многие аспекты работоспособности можно проверить только под объемной, для микроконтроллеров, нагрузкой. Это привело к необходимости на первом этапе проекта не только доработать архитектурные решения, но и написать и отладить большой объем кода, что сделало график работы первого этапа очень напряженным, помимо сложноорганизованных системных модулей, предстояло разработать прикладные драйвера для множества сенсоров и актуаторов.

3.3 ЗАДАЧИ ВТОРОГО ЭТАПА ПРОЕКТА

Главной задачей второго этапа – на данном этапе требовалось разработать архитектуру «верхнего» т.е. серверного уровня фреймворка "Horizon Automation Server", разработать и отладить код основных системных и прикладных служб фреймворка, нивелировать выявленные на старте проекта риски либо найти обходные пути получения требуемого функционала.

Фреймворк серверного уровня – "Horizon Automation Server" предназначен для установки на компактный 64-разрядный одноплатный компьютер с архитектурой ARM или x86 под управлением ОС Linux. Таким образом фреймворк серверного уровня можно развернуть практически на любой аппаратной платформе класса микрокомпьютер, любого производителя. Что делает платформу очень устойчивой к негативным геополитическим факторам.

На самом раннем этапе проработки проекта была принята одна из главных архитектурных концепций построения программного обеспечения ПТК, а именно построение платформы как набора микрослужб, это касается фреймворков и "нижнего" и "верхнего" уровней.

Первой ответственной задачей второго этапа была разработка архитектуры фреймворка "верхнего" уровня "Horizon Automation Server". Принятие микросервисной архитектуры, это важное решение, которое влияет на весь жизненный цикл ПТК, но помимо этого нужно было проработать очень большое количество вопросов, касающихся архитектуры и функционала Hub в целом.

Требовалось принять решения по таким вопросам как:

- выбор среды исполнения JavaScript;
- выбор серверной среды в которой непосредственно будет функционировать фреймворк и в которой пользователь будет вести разработку прикладного кода;
- выбор frontend среды предоставляющая базовый функционал визуализации прикладных, системных и конфигурационных данных обрабатываемых в Hub;
- выбор СУБД и типов коллекций базы данных в которой будут сохраняться как все данные обрабатываемые в Hub так и конфигурации проектов пользователя;
- выбор СУБД предназначенной для оперативного, высокоскоростного обмена данными между ядром фреймворка и сторонними информационно-измерительными системами;
- выбор основной системы логгирования, способной детально хранить пакеты сообщений как приходящих от "нижнего" уровня, от ПЛК и различного рода иных аппаратных источников и компонентов конечного прикладного аппаратного обеспечения (например сборщиков сигналов сторонних производителей, компонентов сетевого обеспечения, источников бесперебойного питания и т.п.);
- выбор сторонней профессиональной системы визуализации, обладающей профессиональными возможностями в области сбора и отображения метрик

технологических процессов обрабатываемых в Hub, обладающая развитыми средствами визуализации, различного рода процессов, имеющую в своем составе развитую систему алертов;

- выбор служб выступающих общепризнанными в системах автоматизации IoT, "Умный дом" и др.;

- выбор системного компонента обеспечивающего возможность организации на базе Hub Wi-Fi и Ethernet хот-спота и маршрутизатора, что необходимо для подключения и взаимодействия с ПЛК по топологии "звезда";

- выбор системного компонента обеспечивающего динамическое присвоение ПЛК и иным сторонним источникам IP адресов;

- выбор системного компонента обеспечивающего функционал службы доменных имен, что позволит достоверно работать с множеством ПЛК и иными сторонними источниками в удобном человеко-читаемом формате при обращения к данным ресурсам;

- выбор системного компонента предоставляющего сервис точного времени ПЛК и иным сторонним источникам, а также обладающего способностью самостоятельно синхронизироваться от сторонних эталонов времени;

- выбор системного компонента предоставляющего сервис получения всех основных данных о состоянии Hub, таких как температура центрального процессора, загрузка ядер ЦП, количество оперативной памяти занятой/свободной, свободное/занятое место на накопителе, загрузка сетевых интерфейсов и т.д. и т.п.;

После того как все решения по вышеописанным пунктам приняты, необходимо выполнить задачи по кастомизации выбранных служб, написать и отладить драйвера для интеграции данных компонентов с ядром серверного фреймворка.

3.4 АРХИТЕКТУРНЫЕ РЕШЕНИЯ УРОВНЯ ПЛК

В соответствующих разделах данного документа, будут подробно рассмотрены разработанные системные и прикладные службы фреймворка "Horizon Automation PLC" и то, как как они взаимодействуют между собой и с внешними информационно-измерительными системами.

В данном разделе будут перечислены службы, их назначение, а также требования к аппаратной платформе, которая были нами отобраны для аппаратной основы ПЛК.

Исполнительная среда фреймворка – Espruino является open source продуктом. Все взаимодействие и разработка прикладного кода в рамках фреймворка ведется на языке JavaScript, с открытым стандартом.

В Приложении №1 представлена структурная схема фреймворка "Horizon Automation PLC". Рассмотрим подробнее службы входящие в него:

Network – системная служба обеспечения связи по стандартам Wi-Fi и Ethernet.

WS Server – служба, реализующая сервер с использованием протокола WebSocket/LHP.

MQTT Gateway – служба, реализующая взаимодействие с MQTT брокером.

Proxy WS – прокси служба WS Server.

Proxy MQTT – прокси служба MQTT сервера.

Application Bus – служба организации сообщений между системными службами ПЛК.

RouteREPL – служба контролирования функционального компонента Espruino – REPL.

DeviceManager – служба, координирующая работу сенсоров и актуаторов.

Process – центральная служба, инициализирующая фреймворк.

Logger – Служба сбора, хранения, ретрансляции логов системы.

Драйвера сенсоров/актуаторов – прикладные компоненты поддержки конкретных моделей сенсоров и актуаторов.

Подробно реализация как системных, так и прикладных модулей рассмотрена далее в соответствующих разделах.

Перед тем как перейти к детальному рассмотрению реализации фреймворка и его составляющих частей, необходимо сделать важное примечание. Перед командой разработчиков стояла задача не просто разработать и отладить все перечисленные компоненты, а сделать это фактически дважды! Так проектом предусмотрена мультиплатформенность, а именно работа фреймворка минимум на двух не совместимых архитектурах ARM и TENSILICA XTENSA, то объем работы над многими модулями фактически удваивался.

Для изготовления прототипов ПЛК, в качестве вычислительного ядра было принято решение использовать существующие микроконтроллерные

модули с микроконтроллером на базе 32-х битного ядра ARM Cortex M4, непосредственно испытания проходили на контроллерах STM32F405, STM32F407, STM32F411.

Как было указано выше программное обеспечение фреймворках является мультиплатформенным и на данный момент может работать на двух совершенно различных платформах, в качестве второй при прототипировании использовались микроконтроллерные модули с микроконтроллером на базе 32-битного ядра Tensilica Xtensa LX6, непосредственно испытания проходили на контроллерах ESP32-D0WDQ6.

3.5 АРХИТЕКТУРНЫЕ РЕШЕНИЯ УРОВНЯ HUB

В соответствующих разделах данного документа, будут рассмотрены разработанные системные и прикладные службы серверного фреймворка, и то, как они взаимодействуют между собой и с внешними информационно-измерительными системами. В Приложении №22 представлена структурная схема фреймворка "Horizon Automation Server".

В данном разделе будут кратко перечислены эти службы, а также требования к аппаратной платформе, которая были нами отобраны для изготовления прототипов Hub.

Перечень системных и прикладных служб Hub разработанных в ходе реализации проекта в соответствии с архитектурой ПТК:

1. **Process** – центральная служба, инициализирует фреймворк;
2. **Logger** – служба логирования;
3. **ProxyLogger** – прокси служба Logger;
4. **DeviceManager** – служба координатор служб-каналов;
5. **ChannelSensor** – служба-каналов типа сенсор;
6. **ChannelActuator** – служба-каналов типа актуатор;
7. **ProxyChannel** – прокси служба ChannelSensor и ChannelActuator;
8. **WSClient** – служба драйвер источников типа ПЛК Horizon;
9. **ProxyWSClient** – прокси служба WSClient;
10. **MQTTClient** – служба драйвер источников типа broker MQTT;
11. **ProxyMQTTClient** – прокси служба MQTTClient;
12. **MQTTGateway** – служба, прокидывающая данные в broker MQTT;
13. **ProxyMQTTGW** – прокси служба MQTTGateway;

Далее представлен перечень сторонних служб, которые были отобраны в соответствии задачами этапа, рассмотренными выше и с которыми были выполнены работы по интеграции:

1. **Node.js** – серверная исполнительная среда исполнения программ JavaScript и среды потокового программирования Node-Red;
2. **Node-Red** - среда потокового программирования и визуализации;
3. **MongoDB** – NoSQL документо-ориентированная СУБД;
4. **Redis** - NoSQL структурированных хранилищ данных в памяти СУБД;
5. **Graylog** - сервер логгирования;
6. **Grafana** - среда визуализация с подсистемой алертов;
7. **Mosquitto** - брокер MQTT;
8. **MQTTX** - web клиент MQTT;
9. **RasAP** - служба реализации точки доступа Wi-Fi "hot spot";
10. **Traefik** - реверс прокси;
11. **Dnsmasq** – DHCP и DNS сервер;
12. **NTP** - сервер точного времени;

3.6 ОПРЕДЕЛЕНИЕ ОСНОВНЫХ РИСКОВ ПРОЕКТА

На начальном этапе реализации этапа проекта перед разработчиками стояла задача наиболее реалистично и точно определить риски, которые будут стоять на пути успешной реализации проекта, и подготовить набор задач направленных в рамках НИОКР на подтверждения рисков или вычеркивания их из списка «угроз» реализации проекта.

В списке рисков на первом месте нами была поставлена угроза нехватки оперативной памяти при реализации фреймворка "Horizon Automation PLC" который будет непосредственно работать на микроконтроллере.

Требование к мультиплатформенности ПЛК, привело к специфичным требованиям в части аппаратного обеспечения. Оно конкретизировалось в том, что наш фреймворк должен иметь возможность выполняться на микроконтроллерах архитектуры ARM и микроконтроллерах архитектуры TENSILICA XTENSA. Это совершенно разные микропроцессорные архитектуры, несовместимые по системе команд и аппаратно. В дальнейшем запланирована адаптация фреймворка под архитектуру RISC-V.

Микроконтроллеры семейства ARM, TENSILICA XTENSA, так же, как и все остальные присутствующие на рынке обладают известным свойством – их аппаратные ресурсы богаты функциональными блоками, но обладают очень скромными ресурсами в части ПЗУ и ОЗУ. И как правило данный ресурс определен на уровне аппаратных свойств микроконтроллера. Это отличает микроконтроллеры от их старших братьев – микропроцессоров.

На момент старта проекта исполнительная среда Espruino на базе которой реализуется, ПО микропроцессорной части нашего фреймворка поддерживает микроконтроллеры ARM семейства Cortex M4, TENSILICA XTENSA семейства L106, LX6. Объем доступной оперативной памяти у данных микроконтроллеров колеблется в районе 80...500 кБ. Ситуация с ПЗУ рисков практически не представляет, т.к. даже у самых нишевых представителей вышеуказанных семейств микроконтроллеров объем встроенной в кристалл флэш памяти может достигать мегабайт и более, что для современных задач, выполняемых на микроконтроллерах, хватает как правило с запасом. К сожалению, такого с уверенностью нельзя сказать о ОЗУ. Пятьсот килобайт — это практически потолок встроенной оперативной памяти для современных микроконтроллеров указанных семейств.

Для программного обеспечения выполняемых на системах с исполнительными средами, такими как Espruino (язык Javascript), Micropython (язык Python) расход памяти происходит намного быстрее чем для таких языков и сред как C++. Это обратная сторона их достоинств перед такими низкоуровневыми языками как C/C++. В связи с этим практически невозможно, на начальном этапе предсказать сколько памяти останется доступной пользователю, после развертывания Espruino и нашего фреймворка. Именно этот показатель был определен командой как главный риск проекта.

Для нивелирования риска, к сожалению, существует только один путь – разработать и протестировать фреймворк в объеме как можно ближе к его конечному виду. Тогда при тестах можно будет с уверенностью определить сколько останется оперативной памяти доступной пользователю и является ли такой показатель коммерчески успешным, или понадобится решать проблему с нехваткой ОЗУ.

В результате данной ситуации у нас сложился план разработки первого этапа проекта, он требовал провести огромную работу, а именно не только разработать архитектурные решения, но написать большой (как можно более большой) модулей фреймворка. Сюда входит как модули системных служб, так и драйвера большого числа сенсоров и актуаторов поддержку которых необходимо обеспечить на первой фазе проекта.

Далее будут подробно рассмотрены фазы реализации задач по разработке ПО системных модулей, а также прикладных драйверов сенсоров и актуаторов поддержка которых идет из «коробки» вместе с нашим фреймворком, также будут представлены выводы как локальные по успешности реализации ПО модулей, так и вывод по отношению ситуации с ОЗУ.

Следующим по значимости риском был определен риск проблем одновременной работы сетевых служб фреймворка которые должны быть разработаны для коммуникаций. Такими службами должны были стать служба, обеспечивающая внешним серверам подписываться на практически любые события и данные генерируемые внутри нашего фреймворка. Такая служба должна базироваться на связке из – WebSocket сервера, протокола нашей разработки LHP, на базе JSON структур и трех служб которые должны обслуживать подписки от внешних серверов на услуги, предоставляемые микропроцессорной системой, параллельно с данным протоколом фреймворк должен обеспечить работу MQTT сервера для объёма данными с внешними серверами по совершенно иному протоколу MQTT.

Основной риск здесь связан с особенностями реализации сетевого интерфейса Wi-Fi, который в нашем проекте определен как основной для обеспечения связи с внешним миром (но проблемы описанные далее относятся и к интерфейсу Ethernet, поддержка которого также обеспечивает наш фреймворк).

Суть проблемы в том, что сетевой стек протоколов TCP/IP весьма сложен и требует для своей работы производительного процессора и значительный объема памяти, к этому добавляется стек Wi-Fi который также требует значительных ресурсов для своей работы. В связи с этим разработчики микроконтроллеров идут на различные ухищрения и оптимизации, которые ведут к определенной ущербности работы сетевых служб на скромных ресурсах микроконтроллеров. Первым следствием таких мероприятий является сильно ограниченное количество сетевых соединений, одновременно поддерживаемых платформой, на большинстве современных микроконтроллерах эта величина равняется 3-5.

Вторая проблема заключается в том, что микроконтроллеры ARM серии Cortex M4, которые поддерживает Espruino, практически не имеют в своем составе радиомодуль и соответственно поддержки Wi-Fi, а это означает что для проекта необходимо подобрать внешнюю поддержку Wi-Fi которая будет иметь неизвестный интерфейс и протокол взаимодействия, такая ситуация привносит неопределенности.

Вторая поддерживаемая нами архитектура TENSILICA XTENSA семейства LX6, в лице кристалла ESP32 напротив имеет на борту микроконтроллера радиомодуль и сетевой стек, но проблема ограниченного количества соединений – 5 там также присутствует. К ней прибавляется проблема отбираемой оперативной памяти для нужд Wi-Fi, из и так не очень большого объема памяти – порядка 500 кб.

Для нивелирования риска необходимо разработать все модули участвующие в формировании процесса обмена информацией, разработать временные средства на стороне сервера (основной объем разработок серверных компонентов предусмотрен вторым этапом проекта) и протестировать множественное соединение одновременно по двум прикладным протоколам WebSocket/LHP и MQTT. Разработка соответствующих служб и результаты тестирования приведены далее в отчете.

Третьим по значимости риском, был определен риск не реализации перехвата управления подсистемой REPL Espruino. Функция «переноса» ввода команд REPL на сервер, а также получения отклика микропроцессорной системы на сервере является ноу-хау проекта и невозможность реализовать такой функционал нанес бы ущерб коммерческой привлекательности разрабатываемой платформе.

Основная проблематика реализация такого функционала состояла в том, что, перехват управления REPL это фактически в некоторой степени хакинг исполнительной системы Espruino. Речь идет о этичном хакинге, т.е. допустимом изменении поведения системы. Но такая возможность не документирована и изначально не было найдено информации как это можно осуществить. Нивелирование риска возможно только попыткой реализовать функцию на практике.

4. ЭТАП 1. РАЗРАБОТКА ФРЕЙМВОРКА "HORIZON AUTOMATION PLC"

4.1 РАЗРАБОТКА МОДУЛЯ NETWORK

В ходе разработки фреймворка "Horizon Automation PLC" требовалось создать службу, обеспечивающую передачу информации по сети с использованием протокола TCP/IP. Служба должна поддерживать физические среды передачи WiFi и Ethernet.

В ходе выполнения задачи был разработан формат конфигурационного файла, работающий в тандеме со службой Network, в котором пользователь определяет сетевые параметры, а также дополнительные настройки, такие как использование статического IP-адреса, настройки шин и включение режима сканирования и т.д. Получая конфигурацию из этого файла, происходит попытка установить соединение к выбранной точке доступа указанным методом. Служба также учитывает особенности чипа, на котором запущен фреймворк, в случае запуска WiFi - в архитектуре TENSILICA ESP32 предусмотрен программный модуль, автоматически определяющий шину и её характеристики при подключении к беспроводной сети.

При инициализации служба выполняет следующие шаги:

1. Чтение конфигурации из файла и определение среды передачи - WiFi или Ethernet;
2. Настройка аппаратной шины, если требуется;
3. Если используется WiFi и в конфигурации указано сканирование, то производится поиск сетей для подключения;
4. Осуществляется попытка подключения к выбранной сети по указанным в конфигурации параметрам.

При успешном подключении управление передаётся второстепенным службам, зависимым от сетевого соединения - устанавливают имя ПЛК по протоколу mDNS, а также синхронизируют время по NTP. В Приложении №2 представлен фрагмент программного обеспечения модуля Network.

Вывод: разработка модуля для обеспечения стабильного канала связи для передачи данных платформы успешно выполнена. Отладка модуля велась на двух архитектурах: ARM и TENSILICA ESP32. На обеих архитектурах код работает без нареканий.

4.2 РАЗРАБОТКА МОДУЛЯ WSSERVER

В ходе разработки фреймворка "Horizon Automation PLC" требовалось создать службу, обеспечивающую передачу данных между ПЛК и Hub по протоколу WebSocket как одному из самых распространённых и универсальных.

Таким образом при решении задачи на уровне ПЛК реализован WebSocket сервер. Служба хранит информацию о подключенных клиентах и передаёт на них данные, или принимает команды с клиентов.

WebSocket сервер хранит в себе массив подключенных к нему клиентов. Клиенты заносятся туда при подключении к серверу, и удаляются оттуда при разрыве соединения. Этот массив используется для отправки сообщений клиентам. Саму отправку запрашивает уже ProxyWS, передавая необходимые данные и список клиентов-подписчиков, кому эти данные предназначены. В Приложении №4 представлен фрагмент программного обеспечения модуля WSSERVER.

Вывод: разработка модуля, реализующего часть протокола WebSocket для передачи данных между ПЛК контроллерами и Hub успешно выполнена, требуемые характеристики достигнуты. Отладка модуля велась на двух архитектурах: ARM и TENSILICA, на обеих архитектурах код работает без замечаний.

4.3 РАЗРАБОТКА МОДУЛЯ PROXYWS

В ходе реализации фреймворка "Horizon Automation PLC" требовалось разработать службу, обеспечивающую интеграцию WSServer с системными службами фреймворка, т.е. в соответствии с архитектурными решениями требовалось разработать прокси службу для WSServer.

Служба реализует функционал посредника между службой типа сервер в роли которой выступает WSServer, и системными службами ядра фреймворка "Horizon Automation PLC". Её задача – обеспечение двунаправленной маршрутизации и, обработки сообщений между фреймворками «нижнего» и «верхнего» уровней. Обработка сообщений включает в себя работу с LHP-протоколом.

При старте фреймворка служба создается при инициализации WSServer и ожидает сообщений от Hub, которые будут поступать через WSServer. Первым поступает запрос к службе DeviceManager на получение списка доступных каналов. DeviceManager направит ответ через обращение к ProxyWS. Затем от Hub поступит запрос на подписку. В результате его обработки ProxyWS сохранит информацию о том, на какие каналы подписался конкретный клиент. DeviceManager запустит циклический опрос каналов, поток сообщений из которого сперва перехватывается службой ProxyWS, форматируется согласно LHP-протоколу и транслируется через WSServer.

Обработка сообщений, поступающих как от служб «Horizon Automation PLC», так и от Hub, выполняется по общему алгоритму, без внимания к содержанию сообщений. Сообщения, которые необходимо передать на Hub, "оборачиваются" в LHP-протокол и передаются WSServer, а нисходящие сообщения наоборот – распаковываются и отправляются на шину данных фреймворка. В Приложении №5 представлен фрагмент программного обеспечения модуля ProxyWS.

Вывод: разработка модуля ProxyWS, реализующего прокси службу WSServer сервера как часть реализации протокола LHP успешно выполнена, требуемые характеристики достигнуты. Отладка модуля велась на двух архитектурах: ARM и TENSILICA, на обеих архитектурах код работает без замечаний.

4.4 РАЗРАБОТКА МОДУЛЯ ROUTEREPL

В ходе реализации фреймворка "Horizon Automation PLC" требовалось разработать службу, обеспечивающую удаленный доступ к подсистеме REPL исполнительской среды Espruino для удаленного управления ПЛК: конфигурации посредством работы с файловой системой, управления и мониторинга выходных сообщений.

В базовой конфигурации Espruino подсистема REPL работает таким образом, что она взаимодействует со стандартной средой программирования и загрузки программ – Espruino Web IDE, которая в общем случае захватывает управление UART портом микропроцессорной платформы, на которой выполняется Espruino и организует трафик между USB портом компьютера, на котором запущена Espruino Web IDE и UART шиной микропроцессорной платформы, по физическому кабелю ограниченной длины стандарта USB.

Перехват и трансляция функционала REPL исполнительской среды Espruino на удаленную консоль позволяет отвязать микропроцессорную платформу от компьютера, которая на начало разработки требовала для осуществления взаимодействия пользователя с подсистемой REPL подключения USB кабелем к компьютеру.

При запуске фреймворка создается служба RouteREPL, в контексте которой запускается TCP-сервер с прослушиванием указанного в конфигурации порта. При установлении подключения служба автоматически выполняет перехват доступа к REPL и маршрутизирует его с открывшимся сокетом. Таким образом, все команды, передающиеся по сети со стороны клиента, направляются в REPL, а вывод REPL передается на сокет. Если пользователь работает в режиме терминала, то вывод будет направляться в его консоль.

Для выполнения отдельных операций, таких как получение списка файлов, загрузка либо скачивание файла, со стороны клиента поступает команда, прерывающая режим терминала и сокет перехватывается уже специализированными функциями-обработчиками, контролирующими входные и выходные данные. В Приложении №6 представлен фрагмент программного обеспечения модуля RouteREPL.

Вывод: разработка службы успешно выполнена, требуемые характеристики достигнуты. Тестирование проводилось на следующих аппаратных платформах – микроконтроллерах ARM и TENSILICA XTENSA при подключении со стороны ПК общего назначения под управлением ОС Linux и Window.

4.5 РАЗРАБОТКА МОДУЛЯ MQTTGW

В ходе реализации фреймворка "Horizon Automation PLC" требовалось разработать службу, выполняющую роль MQTT шлюза и клиента, обеспечивающую работу с источником типа MQTT брокер.

Суть служб типа «шлюз» заключается в предоставлении проброса всех или части информационно-измерительных каналов на внешнюю информационную-измерительную систему. В то же время, конкретно данная служба не является «чистым» представителем типа шлюз так как она обязана не только отображать значения всех или части каналов на стороннюю службу, но и принимать команды на физические актуаторы.

Служба должна обеспечить возможность публикации данные каналов-сенсоров на топики MQTT брокера по протоколу mqtt. Необходима возможность конфигурации сетевых параметров подключения к брокеру (IP адрес, имя пользователя и пароль), а также топиков, с которыми должны сопоставляться каналы сенсоров и актуаторов. Вместе с тем служба должна осуществлять сбор данных с топиков для управления актуаторами.

Служба создается при старте фреймворка и далее ожидает сообщения «connected» от Process, которая служит сигналом к началу работы. По этому сигналу служба выполняет подключение к брокеру и устанавливает обработчики на установление повторного соединения если оно было потеряно в силу деструктивных факторов. Далее служба работает пассивно, выполняя по команде подписку на указанные топики и пропускает через себя поток данных на брокер и с него. Логическое сопоставление имен топиков и каналов берет на себя прокси-служба.

Процесс конфигурации службы организован через существующие механизмы: работа с конфигурационным файлом, которая выполняется через функционал службы Process. Применение конфига позволяет один раз задать настройки сервера и автоматически применять их в прикладных программах. Указание имен топиков интегрировано в конфигурацию каналов датчиков и актуаторов. В Приложении №7 представлен фрагмент программного обеспечения модуля MQTTGW.

Вывод: разработка службы MQTTGW успешно выполнена, требуемые характеристики достигнуты. Тестирование проводилось, с одной стороны на следующих аппаратных платформах – микроконтроллерах ARM и TENSILICA XTENSA, с другой стороны были аппаратные платформы Raspberry Pi 5 и Raspberry Pi Compute Module 4, которые выступали в роли Hub и на которых работал MQTT брокер.

4.6 РАЗРАБОТКА МОДУЛЯ PROXYMQTT

Добавлено примечание ([AM1]): Задвоение нумерации

В ходе реализации фреймворка "Horizon Automation PLC" требовалось разработать службу, обеспечивающую поддержку работы MQTT с ядром фреймворка, т.е. в соответствии с архитектурными решениями требовалось разработать прокси службу для MQTT.

ProхуMQTT выполняет задачу по интеграции MQTT с системными службами фреймворка "Horizon Automation PLC".

Служба выполняет автоматическое сопоставление инициализированных каналов датчиков и актуаторов с топиками подключенного MQTT-брокера. Это позволило выстроить единую систему взаимодействия, где каналы сенсоров и актуаторов логически ассоциируются с топиками брокера, а все коммуникации абстрагируются в рамках унифицированного протокола фреймворка.

Сопоставление каналов топикам реализуется благодаря управлению подписками, что тоже является функционалом службы.

На старте фреймворка служба инициализируется вместе со службой MQTT и направляет на DeviceManager запрос на получение списка доступных каналов, и ожидает ответ, содержащий имена каналов и топики брокера, с которыми их необходимо связать. Из этого ответа и формируется маппинг-таблица. Службе MQTT делегируется выполнить подписку на топики, связанные с каналами-актуаторами. Эти действия делает возможным автоматическое реагирование на обновление значений каналов-сенсоров или топиков, служащих управляющим интерфейсом актуаторов. Дальнейшая работа ProхуMQTT заключается в направлении потока данных, поступающего от DeviceManager по шине фреймворка, на MQTT, который уже выполняет запись на брокер, а также в реагировании на сообщения службы MQTT и формировании управляющих команд каналам-актуаторам.

Таким образом, после выполнения инициализационных процессов, обновление показаний на определенных топиках приведет к изменению состояния канала-актуатора. А сообщение считанное с канала-сенсора будет отражено в соответствующем топике MQTT-брокера. В Приложении №7 представлен фрагмент программного обеспечения модуля ProхуMQTT.

Вывод: задача по разработке службы ProхуMQTT успешно выполнена. Создан механизм двунаправленной маршрутизации сообщений между системными службами и MQTT Server, что обеспечило масштабируемость системы и более «чистую» архитектуру в целом. Были решены ключевые задачи по интеграции с DeviceManager, позволяя транслировать показания датчиков на MQTT-брокер. Функционал был проверен на микроконтроллерах архитектуры ARM и TENSILICA XTENSA.

4.7 РАЗРАБОТКА МОДУЛЯ LHP

Одним из ноу-хау разрабатываемой платформой является поддержка из «коробки» промышленных прикладных протоколов таких как WebSocket и MQTT. Но для инновационной робототехнической платформы этого недостаточно. Учащимся, пользователям, которые только постигают основы embedded, IoT систем, будет проблематично настроить взаимодействие с внешним миром по стандартным протоколам прикладного уровня, потому что в них требуется доработать еще много механизмов для кастомизации своего проекта и доведения его до работоспособного состояния, а работа с протоколами требует серьезных знаний и компетенций.

Помимо описанного минуса, был и прямой недостаток в решении остановится на использовании «чистых» стандартов в области протоколов, таких как WebSocket, MQTT.

Если рассмотреть протокол MQTT, то он де-факто не предназначен для передачи системной и служебной информации. И совершенно не способен реализовать поддержку проброса функционала подсистемы REPL на сторону сервера Node-Red, что является еще одним ноу-хау. Что касается WebSocket, то он весьма гибкий, но это и является его минусом в попытке применить в чистом виде в такой платформе как наша.

Каждый пользователь будет пытаться передавать данные и пытаться получить их совершенно не стандартизированным способом, что приведет к хаосу и неработоспособности платформы.

Исходя из этого мы пришли к необходимости разработать свой протокол взаимодействия с платформой, который станет основным для коммуникации с платформой и через который можно будет получить 100% услуг, которые может предоставить платформа.

Были выставлены следующие требования к протоколу:

- протокол должен базироваться на JSON структурах и легко конвертироваться в объекты JavaScript;
- иметь человекочитаемую структуру для визуального контроля пользователем;
- учитывать все необходимые нюансы архитектуры «нижнего» уровня "Horizon Automation PLC", особенно архитектуру системной службы SensorManager, архитектуру SensorArchitect, а также все системные поля, которые существуют в «нижнем» уровне платформы;
- обладать большим динамическим диапазоном, т.е. способностью к расширению. В дальнейшем количество топиков и служб, которые потребуются, обслуживать будет только увеличиваться, протокол обязан безболезненно поддержать такое расширение;
- для оптимизации объема передаваемого трафика протокол должен опираться на наличие базы данных на стороне сервера для полной реконструкции принимаемых данных, т.е. основной режим обмена сообщениями – сокращенный набор данных, который за счет

продуманной архитектуры «нижнего» и «верхнего» уровней должен уметь восстанавливать всю недостающую информацию из базы данных.

Вывод: в результате работы над модулем был разработан и протестирован – «Легкий Протокол Горизонт» (LHP). Протокол прекрасно прошел тестирования, совместно с серверными службами Node-Red была проверена двухсторонняя доставка сообщений REPL, сбор данных с сенсоров «нижнего» уровня, синхронизация системных параметров. Модуль протокола разработан и полностью соответствует запланированным параметрам. В Приложении №9 представлен фрагмент программного обеспечения модуля LHP.

Вывод: разработка модуля, реализующего протокол LHP успешно выполнена, требуемые характеристики достигнуты. Отладка модуля велась на двух архитектурах: ARM и TENSILICA, на обеих архитектурах код работает без замечаний.

4.8 РАЗРАБОТКА МОДУЛЯ PROCESS

В ходе реализации фреймворка "Horizon Automation PLC" требовалось разработать службу, организующую единую точку старта фреймворка и выполняющую первоначальную инициализацию всех служб ПЛК.

В соответствии с принятыми архитектурными решениями такая служба и на ПЛК и на Hub, получила наименование Process.

Запуск службы Process осуществляется благодаря функционалу, предоставляемому исполнительской средой Espruino. Код в файлах с именами .boot0boot3 будет безусловно выполняться при старте системы. Поэтому разместив создание объекта службы в одном из таких файлов гарантирует её запуск при включении ПЛК.

Для обеспечения гибкости системы при её инициализации служба Process считывает конфигурационные файлы, определяет какую программу запускать, какие службы инициализировать и какие датчики подключать:

- **init.json** - хранит информацию для инициализации системы, включая имя ПЛК, название запускаемой программы и конфигурации;
- **device.json** - хранит данные о периферийных модулях и запускаемых программах, отсортированных по конфигурациям;
- **system.json** - хранит данные об устройствах, предназначенных для служебных нужд, не предназначен для редактирования;
- **services.json** - хранит конфигурацию служб;
- **network.json** - хранит данные, необходимые для сетевого соединения.

После выполнения базовой конфигурации фреймворка, Process приступает к запуску остальных служб. Службы запускаются в зависимости от установленного в конфигурационном файле приоритета. Созданные объекты помещаются в глобальный объект «H», который также хранит описание службы и её статус.

После того, как все службы запущены, Process с помощью функционала DeviceManager создаёт каналы сенсоров и актуаторов, указанных в выбранной конфигурации из device.json и system.json. Имена объектов-каналов, под которыми созданные объекты помещаются в глобальную область видимости, также берутся из конфигурации. Фреймворк контролирует имена объектов-каналов и не позволяет создавать с одинаковыми именами. При неправильной конфигурации, т.е. в ситуации когда пользователь по какой то причине дал одинаковые имена разным каналам, служба проигнорирует создание канала дубликата и выдаст пользователю сообщение об ошибке.

После инициализации запускается задача периодического обновления сведений о системе, синхронизация по времени, а также повторное подключение к сети в случае неудачи или разрыва связи. Интервал составляет

10 секунд. В Приложении №10 представлен фрагмент программного обеспечения модуля Process.

Вывод: разработка модуля Process, успешно выполнена, требуемые характеристики достигнуты. Отладка модуля велась на двух архитектурах: ARM и TENSILICA, на обеих архитектурах код работает без замечаний

4.9 РАЗРАБОТКА МОДУЛЯ DEVICEMANAGER

В ходе реализации фреймворка "Horizon Automation PLC" требовалось разработать службу, которая будет иметь прямой доступ ко всем инициализированным пользователем датчикам и актуаторам, что позволит ей предоставить доступ к измерительно-исполняющему уровню платформы через единый интерфейс, с которым можно взаимодействовать как со стороны ПЛК так и со стороны Hub. Рассмотрим типовой проект с использованием датчиков. Пользователь создает и хранит объекты датчиков в виде независимых переменных. Проблема в том, что при таком сценарии доступ к этим объектам ограничен контекстом среды разработки, и доступ к ним извне (со стороны сервера) технически невозможен, отсутствуют средства автоматического учета созданных датчиков. Из этой проблемы была сформулированы задачи службы:

- 1) Ведение реестра датчиков и актуаторов;
- 2) Автоматический сбор показаний с датчиков для их дальнейшей передачи на сервер;
- 3) Приём команд для управления актуаторами.

Служба создается при старте фреймворка и играет ключевую роль в инициализация каналов устройств: объекты каналов создаются через специализированный метод DeviceManager, в котором выполняется считывание конфигурации устройства, анализ доступности и корректности указанных портов и шин, инициализация и сохранение устройств, предоставление доступа к их каналам.

Далее DeviceManager ожидает от прокси-служб запросы на получения информации о доступных каналах. В ответ на сообщение о подписке внешнего агента на показания датчиков служба запускает циклический опрос каналов-сенсоров, на каждой итерации которого перебирает работающие каналы. При этом для уменьшения трафика данные с каналов, которые существенно не обновились с момента последней итерации, пропускаются.

Вопрос предоставления внешнему агенту доступа к исполнительной среде решен тем, что DeviceManager также принимает от прокси-служб команду на изменение состояния канала-актуатора. Иными словами, модуль позволяет принять с сервера команду, эквивалентную той, что может быть отправлена пользователем прямо через консоль. В Приложении №11 представлен фрагмент программного обеспечения модуля DeviceManager.

Вывод: разработка службы DeviceManager успешно выполнена, требуемый функционал реализован. Тестирование проводилось на микроконтроллерах архитектуры ARM и TENSILICA XTENSA.

5. ЭТАП 2. РАЗРАБОТКА ФРЕЙМВОРКА "HORIZON AUTOMATION SERVER"

5.1 РАЗРАБОТКА МОДУЛЯ PROCESS

В ходе реализации фреймворка "Horizon Automation Server" требовалось разработать службу, организующую единую точку старта фреймворка и выполняющую первоначальную инициализацию всех его служб.

В соответствии с принятыми архитектурными решениями такая служба во всех фреймворках ПТК называется Process.

Запуск службы Process происходит в узле Node-Red при получении LHP сообщения от специально сконфигурированного inject-узла, располагающегося на том же потоке Node-Red. В обязанности службы входят следующие функции:

6. Инициализация всех остальных служб системы;
7. Настройка системы по конфигурации, полученной из БД;
8. Мониторинг состояния фреймворка, служб и каналов;
9. Хранение и предоставление системной службам фреймворка.

Инициализация остальных служб происходит по конфигурации, которая хранится в базе данных. Таким образом пользователю предоставляется полный контроль к конфигурированию фреймворка, и в то же время осуществляется контроль валидности введенных пользователем настроек. Большинство служб поддерживают значения по умолчанию, если поступившие настройки не валидны.

Также в процессе инициализации служба Process создаёт службы каналов - цифровые копии физических датчиков, конфигурации которых также находятся в БД. Каналы активируются при подключении к соответствующим им источникам.

При завершении инициализации фреймворка, к концентратору имеют возможность подключиться программируемые логические контроллеры (источники) по любому поддерживаемому протоколу и по выбранной среде. После подключения данные с сенсоров поступают автоматически. Также имеется возможность подключиться к серверам (например по протоколу websocket) и к шлюзам. Соответствующие службы фреймворка обеспечивают обмен данными и предоставляют к ним доступ пользователю Hub. В Приложении №23 представлен фрагмент программного обеспечения модуля Process.

Вывод: разработка службы Process успешно выполнена, требуемые характеристики достигнуты. Тестирование проводилось на следующих аппаратных платформах – Raspberry Pi 5 и Raspberry Pi Compute Module 4, обе платформы работали под управлением ОС Linux.

5.2 РАЗРАБОТКА МОДУЛЯ PROVIDERMDB

В ходе реализации серверного фреймворка "Horizon Automation Server" требовалось разработать службу providerMDB, задача которой обеспечить полномасштабную поддержку СУБД MongoDB. В задачи службы входит хранение всей конфигурационной информации серверного фреймворка, архивирование «сырых» и обработанных данных от каналов сенсоров, предоставление третьим компонентам системы, таким как Grafana и Graylog интерфейса для получения данных, хранение текущей настроенной конфигурации фреймворка как – «проекта», предоставление пользователю возможности сохранять фактически неограниченно количество «проектов», хранение базы данных по сенсорам и актуаторам поддерживаемым системой и многое другое. Как видно из описания служба ProviderMDB одна из ключевых служб ядра наравне с Process и DeviceManager.

Модуль службы должен был предоставить всем службам фреймворка унифицированный интерфейс взаимодействия с СУБД MongoDB. В зоне ответственности службы «знания» всех NoSQL коллекций СУБД, умение работать с форматами документов, которые хранятся в коллекциях и т.д.

Рассмотрим коллекции, которые предусмотрены проектом и их назначение:

- | | |
|---------------------------|--|
| 1. rawData | – хранит сырые данные каналов; |
| 2. workData | – хранит обработанные данные каналов; |
| 3. archiveData | – хранит архивные данные от любых каналов; |
| 4. loggingData | – резервное место хранения логов; |
| 5. systemData | – хранит конфигурация системных служб; |
| 6. projectData | – хранит информация о проекте в целом; |
| 7. devicesHandbook | – хранит БД по сенсорам и актуаторам. |

Для долговременной стабильной работы Hub, была проведена работа с типами коллекций. Часть из них: rawData, workData имеют тип – timeseries. Данный тип означает, что время жизни документов в таких коллекциях ограничен по времени, при превышении заданного времени документы удаляются. Коллекции rawData, workData могут очень быстро заполняться и если не предусмотреть алгоритм ограничения объема хранимых в этих коллекциях данных, это в итоге неизбежно приведет к переполнению дисков Hub и краху системы.

Остальные коллекции также настроены специфичным образом, они имеют тип: capped. Данный тип означает, что емкость данных коллекций также ограничена, но уже не по времени жизни документов, а объему данных и при превышении объема документы, которые попали в коллекцию первыми, будут уничтожаться, что приведет к поддержанию размера коллекции на одном уровне.

Для общения с другими сервисами фреймворка была разработана системная шина mdbBus, по данной шине отвечает на запросы других сервисов фреймворка.

Для отработки функционала службы был сформирован стенд с несколькими ПЛК (с набором сенсоров), MQTT брокером в качестве дополнительного источника с большим числом каналов. В Приложении №24 представлен фрагмент программного обеспечения модуля ProviderMDB.

Вывод: разработка службы ProviderMDB успешно выполнена, требуемый функционал реализован. Тестирование проводилось на Hub на базе модулей Raspberry Pi 5 и Raspberry Pi Compute Module 4 под управлением ОС Linux.

5.3 РАЗРАБОТКА МОДУЛЯ LOGGER

В ходе реализации фреймворка "Horizon Automation Server" требовалось разработать единую для ПТК, развитую службу логирования.

Служба должна единообразно и эффективно обслуживать сообщения поступающие от контроллеров ПЛК, сторонних аппаратных средств, внутренних информационно-измерительных служб Hub, служб операционной системы Hub, ядра фреймворка, прикладного кода пользователя.

Служба должна обладать развитыми механизмами дифференцирования поступающих сообщений по тем или иным признакам, обладать развитыми возможностями в части поиска, в том числе обладать возможностью полнотекстового поиска на русском языке.

В соответствии с принятыми архитектурными решениями, в случае, когда системная служба фреймворка работает с внешними источниками, реализация службы разбивается на две – целевую и прокси службы. В данном случае это Logger и ProxyLogger. Служба ProxyLogger, будет рассмотрена далее. Рассмотрим подробнее функционал Logger.

Первоначально был отобран профессиональный сервис, который позволял бы в принципе реализовать в разрабатываемой платформе такой функционал. В качестве главного кандидата был отобран сервер логирования Graylog. Это open source продукт Enterprise класса в области логирования. Сервер является трехзвенным продуктом, и состоит из: интегрирующего web сервера Graylog с которым взаимодействует пользователь, специальной службой полнотекстового поиска Elasticsearch, и базы данных MongoDB.

На первом этапе была произведена его кастомизация для работы на выбранной аппаратной платформе. На втором было произведен анализ вариантов интеграции с ядром фреймворка.

При интеграции с фреймворком необходимо было реализовать возможность логирования сообщений не только из ядра фреймворка, но из user space области Node-Red, т.е. области, в которой работает пользователь над прикладным кодом. Еще раз подчеркнем, в соответствии с принятыми архитектурными решениями, служба логирования должна быть единая для всего комплекса это означает, что сообщения от ядра фреймворка ПЛК и user space кода разрабатываемого на стороне ПЛК, а также от любого иного стороннего аппаратного обеспечения используемого в прикладном проекте, и от системных и прикладных служб должны использовать механизм логирования разработанный в ходе реализации проекта ПТК.

Данное требование реализовать технически довольно сложно, так как исполнительная среда Espruino опирается на средства логирования языка JavaScript (команда console.log), ядро фреймворка ПЛК также опирается на эти возможности, ОС Linux имеет свои средства логирования, в среде потокового программирования также присутствует еще одна не совместимая система логирования, опирающаяся свои кастомные решения.

Непосредственно, служба Logger обслуживает только сообщения, которые уже попали в ядро фреймворка, это означает что Logger хоть и

реализует задуманный функционал на базе внешнего сервера логирования Graylog, но воспользоваться им может только непосредственно службы фреймворков Hub и ПЛК.

В ходе разработки службы Logger, она была интегрирована в ядро фреймворка, что наделило ее возможностью коммуницировать со всеми другими службами фреймворка. Для, отправки сообщений на Logger их необходимо отправить на специальную, спроектированную под Logger, системную шину logBus. При получении сообщения служба подготавливает пакет в формате GELF – родной протокол Graylog и отправляет полное сообщение с добавленными метаданными на специальный input Graylog. Это дает пользователю использовать обширные поисковые возможности Graylog.

Конфигурация необходимая для подключения к серверу Graylog, в соответствии с архитектурными решениями проекта, хранится в БД MongoDB и на старте системы извлекается из БД посредством запроса к службе работающей непосредственно с БД – ProviderMDB.

Пользователь может настроить Logger так, чтобы тот работал в особом debug режиме. В таком режиме служба захватывает все сообщения со всех существующих системных шин фреймворка и безусловно их логирует.

В данном режиме происходит значительная нагрузка на аппаратную платформу т.к. на Graylog от ПТК "сыпятся" от сотен до десятков тысяч сообщений в минуту, плотность трафика определяется масштабом прикладного проекта, количеством подключенных ПЛК к Hub. Такой режим позволяет сопоставить и проанализировать колоссальный объем информации и выявить "баг", но не стоит его включать на продолжительное время и тем более держать постоянно включенным. В данном режиме быстро перезапираются хранилища сообщений Graylog, что также ведет к износу flash памяти и небольшой глубине хранения логируемых сообщений. В Приложении №24 представлен фрагмент программного обеспечения модуля Logger.

Вывод: разработка службы успешно выполнена, требуемые характеристики достигнуты. Тестирование проводилось на следующих аппаратных платформах – Raspberry Pi 5 и Raspberry Pi Compute Module 4, обе платформы работали под управлением ОС Linux.

5.4 РАЗРАБОТКА МОДУЛЯ PROXYLOGGER

В ходе реализации фреймворка "Horizon Automation Server" требовалось разработать специальную службу, позволяющую пользователю Hub логировать сообщения, возникающие в разрабатываемом прикладном коде, т.е. в user space среды Node-Red.

Ранее, при описании службы Logger было показано, что прямого доступа к системным службам и шинам у пользователя нет. Это сделано, во-первых, из-за реализации мер информационной безопасности, а также для того, чтобы пользователю не приходилось изучать обширную документацию внутреннего устройства ядра фреймворка и организацию взаимодействия системных служб.

В таких случаях штатным решением разрабатываемого ПТК является разработка специализированной службы класса прокси, которая выступает посредником между внешними источниками и системными службами ядра фреймворка. В данном случае такой службой выступает – ProxyLogger.

ProxyLogger обеспечивает шлюзование сообщений от прикладного кода пользователя, разработанного в user space, к системной службе Logger, упрощая взаимодействие пользователя со службой логирования.

ProxyLogger является частью системного интерфейса и обеспечивает функциональную прослойку между внутренними, системными уровнями ядра фреймворка и user space пространством Node-Red. Сообщения от прикладного кода пользователя проходят на Logger через прокси службу ProxyLogger. Такая техническая реализация соответствует принятым архитектурными решениями и требованиями безопасности. Данный подход позволил унифицировать реализацию функционала службы Logger для разных внутренних и внешних источников.

Служба ProxyLogger получает от пользовательского кода, минимально подготовленного им сообщение, которое требуется логировать. После получения производится проверка на валидность, конвертация во внутренний протокол LHP и отправку в службу Logger, которая в свою очередь, конвертирует сообщение в родной для Graylog формат – GELF и уже отправляет непосредственно на специально созданный для этого input Graylog сервера.

При получении сообщения от пользовательского кода ProxyLogger инжектирует в транслируемое сообщение важные атрибуты такие как: имя пользовательского узла Node-Red, из которого была произведена отправка сообщения для логирования, имя потока Node-Red, на котором этот узел располагается. Сообщения, переданные через ProxyLogger, имеют специальную маркировку источника, что позволяет чрезвычайно быстро и эффективно фильтровать, и искать, средствами Graylog сообщения с требуемыми атрибутами. В Приложении №26 представлен фрагмент программного обеспечения модуля ProxyLogger.

Вывод: разработка службы успешно выполнена, требуемые характеристики достигнуты. Тестирование проводилось на следующих

аппаратных платформах – Raspberry Pi 5 и Raspberry Pi Compute Module 4, обе платформы работали под управлением ОС Linux.

5.5 РАЗРАБОТКА МОДУЛЯ DEVICEMANAGER

В ходе реализации фреймворка "Horizon Automation Server" требовалось разработать службу, выполняющую управление службами-каналами Hub и осуществляющую координацию взаимодействия между каналами и другими службами ядра фреймворка.

Служба с аналогичным функционалом была ранее разработана при работе над фреймворком ПЛК "Horizon Automation PLC". Более того между службой на стороне ПЛК и Hub есть связь. Поэтому при разработке службы на стороне Hub были учтены опыт и структура ранее разработанной службы.

Служба играет ключевую роль в инициализации каналов данных: DeviceManager анализирует доступные источники и в зависимости от их протокола (mqtt, ws/lhp) формирует запросы к соответствующим прокси-службам для получения информации о доступных каналах. На основе этой информации автоматически активируются службы-каналы, виртуальные двойники сенсоров и актуаторов. Служба отслеживает статус всех инициализированных каналов, уведомляя службу Process об их готовности к началу работы.

Далее служба, вновь через прокси, отправляет запросы на подписку, обеспечивая получение данных от всех доступных источников. Полученные данные о каналах отражаются в state-объектах, которые поддерживают актуальное состояние источников в реальном времени.

Таким образом DeviceManager оптимизирует работу с каналами, исключая необходимость параллельной обработки сообщений каждой отдельной службы-канала.

Благодаря работе DeviceManager достигается централизованное взаимодействие с источниками и каналами данных, автоматическая инициализация виртуальных устройств на основе данных от источников, снижение нагрузки на систему за счёт оптимизации коммуникаций через единый координирующий модуль, актуализация информации о каналах в реальном времени и мониторинг их статусов. В Приложении №27 представлен фрагмент программного обеспечения модуля DeviceManager.

Вывод: разработка службы успешно выполнена, требуемые характеристики достигнуты. Тестирование проводилось на следующих аппаратных платформах – Raspberry Pi 5 и Raspberry Pi Compute Module 4, обе платформы работали под управлением ОС Linux.

5.6 РАЗРАБОТКА МОДУЛЯ CHANNEL

В ходе реализации фреймворка "Horizon Automation Server" требовалось разработать службу, отвечающую за создание и работу информационно-измерительных каналов (далее в этом разделе – каналов).

Концепция архитектуры ПТК "Horizon Automation" в части обработки данных, поступающих от физических сенсоров и актуаторов, заключается в том, чтобы не смотря на различие и сложность внутреннего устройства между различными сенсорами и актуаторами – сформировать "внутри" фреймворка единообразные каналы, измерительные в случае сенсоров и управленческие в случае актуаторов.

Современные сенсоры и актуаторы, преобразующие физическое воздействие в пропорциональный аналоговый или цифровой сигнал – устройства довольно сложные. Зачастую сенсоры имеют не один, а два и более измерительных трактов измерения разных физических величин. Измерения в этих трактах построены на разных физических принципах, имеют различные параметры времени преобразования.

Таким образом невозможно найти общность даже среди сенсоров, измеряющих один и тот же физический параметр, но произведенный разными производителями. Помимо различий во внутреннем устройстве измерительного тракта, современные сенсоры и актуаторы являются устройствами цифровыми, которые зачастую передают данные в микроконтроллер не в виде аналогового сигнала и даже не цифрового, а по цифровым шинам, с использованием того или иного цифрового протокола.

Чтобы нивелировать все указанные отличия, и предоставить пользователю простой и лаконичный интерфейс для работы с сенсорами и актуаторами, архитектурными решениями ПТК было предусмотрено разработка служб которые должны превратить сложно организованный сенсор/актуатор в относительно простой объект (в терминологии языка JavaScript). Такие объекты получили название в терминологии фреймворков – информационно-измерительный канал, или просто канал.

Таким образом в отношении сенсоров и актуаторов была реализована концепция цифровых двойников. Каналы в ПТК, как на уровне ПЛК, так и на уровне Hub ведут себя как двойники физического устройства. Они имеют те же что и физический прибор атрибуты, реагируют на изменение измеряемого параметра в реальном масштабе времени и т.д.

При этом данная концепция предусматривает "расщепление" реального сенсора/актуатора на единичные измерительные/управленческие сущности. Для понимания приведем пример некоторого цифрового сенсора, который измеряет давление и температуру окружающей среды, передает их по цифровой шине. Измерение давления и температуры внутри сенсора производится разными рецепторами, скорость преобразования величин сильно различается, поэтому различен и темп, с которым сенсор может отдать значение давления и температуры. В нашей платформе данному сенсору будут сопоставлены два независимых канала с которыми и будет фактически

работать пользователь. При этом он, например может работать только с одним каналом, а второй даже не создавать, допустим его интересует только давление.

При этом реальный протокол сенсора может требовать обязательного считывания обеих параметров, это может быть связано с его внутренним физическим устройством. Все эти вещи будут нивелированы концепцией цифрового двойника сенсора.

Мониторинг такого канала равноценен мониторингу его физического источника. А команда, направленная на канал, сопоставленный с актуатором, в реальном времени приведет к изменению состояния реального актуатора, например команда, отправленная на канал, отвечающий за сервопривод, приведет к изменению положения его ротора.

Важный момент, который также был реализован в рамках концепции ПТК, это то, что каждый созданный канал в Hub это – служба в терминологии и понимании фреймворка ПТК. Таким образом канал не объект, а субъект системы, со своим жизненным циклом, интерфейсом взаимодействия, своими системными шинами по которым происходит коммуникация с другими службами.

Отметим, что такая концепция соответствует каналам в Hub, т.е. в фреймворке "Horizon Automation Server". Каналы в ПЛК тоже являются цифровыми двойниками своих физических прототипов, но организация каналов с точки зрения архитектуры программного обеспечения, там несколько проще и отличается от реализации Hub, это в первую очередь связано с тем что на уровне ПЛК происходит как правило прямое взаимодействие с физическим сенсором/актуатором, в то время как Hub изначально как правило имеет дело с цифровым источником, в котором фаза декомпозиции сенсора/актуатора способного отдать данные физического параметра уже выполнена и Hub как правило на входе получает данные с источника как единичный измерительный или управленческий канал.

Еще одна причина, по которой реализация отличается – сильные отличия в аппаратных возможностях ПЛК и Hub. В ПЛК вычислительные и другие возможности на несколько порядков меньше аналогичных в Hub. Поэтому там при реализации каналов применялись различные оптимизации. Тем не менее главное свойство каналов – выступать в роли цифрового двойника реализована аналогично и в ПЛК и в Hub, при этом работа пользователя с каналами везде ведется одинаково.

Для реализации данной концепции была разработана служба Channel, предоставляющая унифицированный подход в работе с каналами. Конфигурации каналов в рамках прикладного проекта выполняется перед запуском фреймворка, конфигурации каналов Hub хранятся в БД.

В БД, в формате JSON, создаются записи, соответствующие каналам, каждая из которых содержит необходимые сведения: тип – сенсор/актуатор, источник из которого брать данные сенсора/актуатора, имя, номер, артикул и т.д. Службы-каналы инстанцируются сразу при старте фреймворка, но

активация, т.е. старт физического обмена с источником данных происходит позже, в соответствии с алгоритмом запуска фреймворка.

Поведение и назначение службы канала зависит от её типа: каналы-сенсоры служат для отражения данных с измерительных устройств, а каналы-актуаторы предназначены для управления, исполнительными устройствами.

Служба Channel обеспечивает абстракцию физического и транспортного уровня, скрывая и особенности взаимодействия с физическим оборудованием, и прикладные протоколы, использованные для коммуникации между сервером и источником. При этом, каждый такой канал является носителем информации о физическом датчике/актуаторе, требуя передавать в конфигурации его имя, номер канала, и ряд других сведений.

Служба обеспечивает возможность индивидуальной настройки канала такие как использованная единица измерения, пороговые значения и др. Это позволяет пользователю использовать встроенные во фреймворк алгоритмы обработки данных каналов для решения прикладных задач.

Разработка службы с таким функционалом позволила достичь унифицированного интерфейса взаимодействия для всех типов источников, масштабируемость системы, возможности неограниченно наращивать количество каналов в прикладном проекте. В Приложении №28 представлен фрагмент программного обеспечения модуля Channel.

Вывод: разработка службы успешно выполнена, требуемые характеристики достигнуты. Тестирование проводилось на следующих аппаратных платформах – Raspberry Pi 5 и Raspberry Pi Compute Module 4, обе платформы работали под управлением ОС Linux.

5.7 РАЗРАБОТКА МОДУЛЯ PROXYCHANNEL

В ходе реализации фреймворка "Horizon Automation Server" требовалось разработать службу, предоставляющую пользователю интерфейс между системной службой Channel и пользовательским кодом, работающим в user space.

Как уже неоднократно ранее было показано, для взаимодействия между ядром фреймворка и кодом, работающим в user space требуется прокси служба, которая обеспечит безопасное взаимодействие.

В результате реализации задачи была разработана ProxyChannel служба, которая является частью системного интерфейса и обеспечивает функциональную прослойку между серверным фреймворком и пространством Node-Red.

ProxyChannel является самостоятельной службой, которой необходима только информация о каналах, хранящаяся у службы Process. В случае необходимости получать данные с определённого канала в user space, пользователь может выбрать узел в потоке Node-Red и «подписать» его. В результате такой подписки данные будут поступать на выход указанного узла, а пользователь может уже перенаправить их по своему желанию.

Служба непрерывно слушает все сообщения, генерируемые каналами, и если сообщение пришло от канала, на которой была осуществлена подписка, служба перенаправляет его на выход соответствующего узла в формате LHP-пакета. Во время подписки пользователь имеет возможность указать какой тип данных он ожидает от канала (значение с датчика, единица измерения, события перехода в определённую зону и т.д.).

Аналогично служба поддерживает возможность работу с каналами актуаторов. Для этого пользователю требуется вызвать специальный метод, где нужно указать имя канала и значение. В случае если канал, с которым работает служба, по какой-то причине становится неактивным – система уведомит пользователя об этом. В Приложении №29 представлен фрагмент программного обеспечения модуля ProxyChannel.

Вывод: разработка службы успешно выполнена, требуемые характеристики достигнуты. Тестирование проводилось на следующих аппаратных платформах – Raspberry Pi 5 и Raspberry Pi Compute Module 4, обе платформы работали под управлением ОС Linux.

5.8 РАЗРАБОТКА МОДУЛЯ WSCLIENT

В ходе реализации фреймворка "Horizon Automation Server" требовалось разработать службу, обеспечивающую работу с источником типа ПЛК, работающими под управлением фреймворка "Horizon Automation PLC".

Служба WSClient осуществляет функцию драйвера для данного типа источника. В задачу службы входит установление соединения с источником, поддержание соединения, установление повторного соединения если оно было потеряно в силу деструктивных факторов. Основная функция службы – шлюзование пакетов данных между компонентами фреймворка "Horizon Automation Server" и контроллерами ПЛК под управлением фреймворка "Horizon Automation PLC". Для обмена данными с ПЛК служба использует протокол LHP.

Служба принимает от других служб фреймворка запросы на передачу данных на выбранный ПЛК, далее она упаковывает данные в LHP пакет с помощью специализированного драйвера. Аналогично работает трансляция данных и в обратную сторону, если от ПЛК приходит пакет с данными, WSClient с помощью специализированного драйвера LHP извлекает их и направляет той службе фреймворка которой они предназначаются.

Как и в случае с другими службами фреймворка, обеспечивающими взаимодействие с источниками данных, WSClient работает в тандеме с прокси службой ProxyWSClient. Задача которой обеспечить интерфейс между WSClient и ядром фреймворка, т.е. с другими системными службами.

Инициацию подключения WSClient к заданным в конфигурации источникам осуществляет служба Process, генерируя в специальное событие на системной шине. С этим событием передаётся информация об источниках, к которым необходимо осуществить подключение. Если по какой-то причине подключиться не удалось, или же соединение оборвалось во время работы, то система уведомит об этом пользователя и направит копию сообщения в системный логгер. В Приложении №30 представлен фрагмент программного обеспечения модуля WSClient.

Вывод: разработка службы успешно выполнена, требуемые характеристики достигнуты. Тестирование проводилось на следующих аппаратных платформах – Raspberry Pi 5 и Raspberry Pi Compute Module 4, обе платформы работали под управлением ОС Linux.

5.9 РАЗРАБОТКА МОДУЛЯ PROXYWSCIENT

В ходе реализации фреймворка "Horizon Automation Server" требовалось разработать службу, обеспечивающую поддержку работы WSCient с ядром фреймворка, т.е. в соответствии с архитектурными решениями требовалось разработать прокси службу для WSCient.

Разработка службы велась с учетом опыта аналогичной службы разработанной на стороне ПЛК в фреймворке "Horizon Automation PLC". Служба реализует функционал посредника между службой типа source в роли которой выступает WSCient источников, и системными службами ядра фреймворка "Horizon Automation Server".

ProxyWSCient обеспечивает:

- Выравнивание коммуникаций: выстраивание единообразного взаимодействия между источниками и системными компонентами, независимо от специфики протокола;
- Управление подписками: обработку запросов на подписку на каналы источников и передачу данных от WSCient в системные службы;
- Унификацию протоколов: абстрагирование протокола LHP, что упрощает интеграцию новых источников в серверный фреймворк.

После старта фреймворка по команде ProxyWSCient через WSCient направляет на источники запросы на получение информации о доступных каналах. Затем служба дожидается ответа и передает полученную информацию в DeviceManager, который в последствии формирует запросы на подписку на эти каналы, что вновь происходит через ProxyWSCient.

Служба предоставляет унифицированный интерфейс для отправки команд на LHP-источники и приёма ответов. Каждое сообщение, направленное на источник, форматируется согласно LHP-протоколу и вкладывается в команду "отправить" WSCient. Таким образом, WSCient отправляет итоговое сообщение на источник, совершенно абстрагируясь от его содержания. Сообщения, исходящие от источников, попадают на Службу через WSCient, после чего они распаковываются и передаются на шину фреймворка от имени ProxyWSCient.

Благодаря тандемной работе со службой ProxyWSServer фреймворка полевого уровня, удается достичь эффекта того, что сложное взаимодействие с системными службами ПЛК скрывается под обменом сообщениями со службой ProxyWSCient.

Вывод: разработка службы успешно выполнена, требуемые характеристики достигнуты. Тестирование проводилось на следующих аппаратных платформах – Raspberry Pi 5 и Raspberry Pi Compute Module 4, обе платформы работали под управлением ОС Linux.

5.10 РАЗРАБОТКА МОДУЛЯ MQTTCLIENT

В ходе реализации фреймворка "Horizon Automation Server" требовалось разработать службу, обеспечивающую работу с источником типа MQTT брокер.

Служба MQTTClient осуществляет функцию драйвера для данного типа источника. В задачу службы входит установление соединения с источником, поддержание соединения, установление повторного соединения если оно было потеряно в силу деструктивных факторов. Основная функция службы – шлюзование пакетов данных между компонентами фреймворка "Horizon Automation Server" и брокерами типа MQTT. Для обмена данными с источниками служба использует протокол mqtt либо mqtt over ws (mqtt на основе websocket). Назначение MQTTClient заключается в том, чтобы связать информационно-измерительные каналы фреймворка, с соответствующими топиками MQTT-брокера. Подписка на эти топики обеспечивает поступление данных от реальных устройств в Hub, что позволяет оперативно получать информацию о текущем состоянии датчиков. В то же время публикация значений в топики, ассоциированные с актуаторами, позволяет динамически изменять их физическое состояние в режиме реального времени.

Как и в случае с другими службами фреймворка, обеспечивающими взаимодействие с источниками данных, MQTTClient работает в тандеме с прокси службой ProxuMQTTClient, которая и выполняет задачу логического сопоставления топиков источников и инициализированных служб-каналов. При этом функционал MQTTClient остается прозрачным: подписка, запись, чтение.

Интеграция с MQTT-брокерами открывает возможность работать с распределенными системами, где данные поступают от множества удаленных устройств.

При запуске фреймворка служба инициирует подключение ко всем зарегистрированным источникам, использующим протокол MQTT. Для каждого источника создается отдельное соединение с брокером, а его состояние отслеживается в реальном времени.

Для обеспечения надежности работы службы предусмотрено автоматическое восстановление соединений в случае их разрыва. При отключении какого-либо источника информация об этом немедленно отправляется в системный компонент SourcesState, который обновляет текущее состояние всех источников, а также логируется соответствующее событие. В Приложении №31 представлен фрагмент программного обеспечения модуля MQTTClient.

Вывод: разработка службы успешно выполнена, требуемые характеристики достигнуты. Тестирование проводилось на следующих аппаратных платформах – Raspberry Pi 5 и Raspberry Pi Compute Module 4, обе платформы работали под управлением ОС Linux.

5.11 РАЗРАБОТКА МОДУЛЯ PROXYMQTTCLIENT

В ходе реализации фреймворка "Horizon Automation Server" требовалось разработать службу, обеспечивающую поддержку работы MQTTClient с ядром фреймворка, т.е. в соответствии с архитектурными решениями требовалось разработать прокси службу для MQTTClient.

ProxyMQTTClient выполняет задачу по предоставлению двухстороннего интерфейса между MQTTClient и системными службами ядра фреймворка "Horizon Automation Server".

Служба преобразовывает данные и команды между службами-каналами и MQTT-брокерами, которые сами по себе являются интерфейсом информационно-измерительных систем. Это позволило выстроить единую систему взаимодействия, где каналы сенсоров и актуаторов логически ассоциируются с топиками брокера, а все коммуникации абстрагируются в рамках унифицированного протокола фреймворка.

Как и остальные прокси службы проекта, ProxyMQTTClient обеспечивает унификацию взаимодействия служб-источников и прикладных служб. Сопоставление каналов топикам реализуется благодаря управлению подписками, что тоже является функционалом служб.

После старта фреймворка ProxyMQTTClient ожидает от DeviceManager запрос на подписку, к которому прилагается список инициализированных служб-каналов. При обработке данного запроса создается внутреннюю маппинг-таблица, которая связывает топики брокера с именами и адресами виртуальных каналов. Это делает возможным получение данных от источников и их последующую обработку в рамках системных служб. Затем ProxyMQTTClient делегирует MQTTClient выполнить подписку на топики, ассоциированные со службами сенсорами. Дальнейшая работа ProxyMQTTClient заключается в форматировании и пересылке сообщений от служб-актуаторов и от службы MQTTClient. При получении сообщения от службы-канала, из маппинг-таблицы считывается имя ассоциированного с ним топика. За этим следует отправка запроса службе MQTTClient на запись в этот топик. При получении сообщения от MQTTClient по топик, переданному в теле сообщения, определяется имя службы-канала, на которую по шине фреймворка направляется сообщение о поступлении новых показаний датчика.

Таким образом, после выполнения инициализационных процессов, обновление показаний на определенных топиках приведет к обновлению значения службы-канала. А команда, переданная на канал-актуатор из userspace инициирует запись в указанный в маппинг-таблице топик брокера. В Приложении №32 представлен фрагмент программного обеспечения модуля ProxyMQTTClient.

Вывод: разработка службы успешно выполнена, требуемые характеристики достигнуты. Тестирование проводилось на следующих аппаратных платформах – Raspberry Pi 5 и Raspberry Pi Compute Module 4, обе платформы работали под управлением ОС Linux.

5.12 РАЗРАБОТКА МОДУЛЯ MQTTGW

В ходе реализации фреймворка "Horizon Automation Server" требовалось разработать службу типа gateway, обеспечивающую работу с источником типа MQTT брокер.

Суть служб типа заключается в предоставлении проброса всех или части информационно-измерительных каналов на внешнюю информационную-измерительную систему. В данном случае в роли такой службы выступает MQTT брокер, а задача службы MQTTGW маппировать значения каналов фреймворка в топики брокера. Еще раз подчеркнём, службы типа gateway работают только "в одну сторону", это означает, что они безусловно отображают значения всех или части каналов на стороннюю службу.

Служба должна обеспечить возможность публикации данные каналов топики MQTT брое по протоколам mqtt либо mqtt over ws (mqtt на основе websocket). Конфигурация в которой описывается сетевые параметры брокера (доменное имя узла, IP адрес, тип протокола по которому необходимо с ним работать и т.п), а также реестр каналов которые должны быть опубликованы, а также наименование корневых веток, которые должны быть созданы службой храниться в БД и предоставляется службой ProviderMDB по запросу MQTTGW. В Приложении №33 представлен фрагмент программного обеспечения модуля MQTTGW.

Вывод: разработка службы успешно выполнена, требуемые характеристики достигнуты. Тестирование проводилось на следующих аппаратных платформах – Raspberry Pi 5 и Raspberry Pi Compute Module 4, обе платформы работали под управлением ОС Linux.

6. ЗАКЛЮЧЕНИЕ

Выводы по итогу реализации проекта. В результате выполнения проекта была спроектирована и разработана архитектура:

1. Программно-технического комплекса "Horizon Automation" в целом;
2. Фреймворка "нижнего" уровня "Horizon Automation PLC";
3. Фреймворка "верхнего" уровня "Horizon Automation Server";

Изготовлены образцы прототипов PLC (контроллеров нижнего уровня) и Hub (контроллер верхнего уровня)

Результаты работы над программируемым-логическим контроллером и фреймворком "нижнего" уровня "Horizon Automation PLC":

С большим перекрытием по запланированному количеству драйверов сенсоров и актуаторов, а также по объему функций и сервисов реализована служба фреймворка "Horizon Automation PLC" в соответствии с определенной архитектурой ПЛК в целом.

Проведены сотни различных тестов по определению работоспособности как отдельных частей фреймворка, так и ПЛК как единицы автоматизации. Получен колоссальный опыт работы с исполнительской средой Espruino. Изучены ее сильные и слабые стороны, проработаны методологии преодоления "узких" мест.

Полностью подтверждена гипотеза, о том, что современный интерпретируемый язык такой JavaScript может быть эффективной основой современных ПЛК, применяемых как в проектах образовательной робототехники, математического моделирования, кибер-физики, информатики. Так и в сферах промышленной автоматизации, системах умный дом, диспетчеризации. При этом подтверждено, что такое решение изначально дает ряд преимуществ теоретически недостижимых с помощью "классических" языков программирования C/C++, долгое время доминирующих в разработке прикладного обеспечения контроллеров, предназначенных для различных сфер автоматизации.

Риски реализации фреймворка "нижнего" уровня, перечисленные в соответствующем разделе, были идентифицированы, после чего были выработаны мероприятия, которые их нивелировали, что подтвердилось тестами. Запланировано мероприятие по предотвращению появления рисков, связанных с потенциальной нехваткой памяти при реализации на ПЛК больших прикладных проектов. Оно заключается в работе по модификации исполнительской среды с тем, чтобы в проектах на микроконтроллерах ARM и ESP32 можно было задействовать современную внешнюю псевдо-статическую память PSRAM.

Множественные тесты ПЛК под управлением, с одной стороны, показали работоспособность и функциональность фреймворка "Horizon Automation PLC", соответствующую ТЗ, но также показало направление дальнейшего развития.

Команда вплотную подобралась к пониманию внутренних механизмов работы исполнительной среды Espruino, и ее ограничений. Намечен план создания своего форка проекта Espruino, с тем чтобы устранить имеющиеся у нее недостатки, наделить новым функционалом, не зависеть от графика выхода обновлений.

Результаты работы над концентратором Hub и фреймворком "верхнего" уровня "Horizon Automation Server":

Объем реализованной работы над фреймворком "верхнего" уровня "Horizon Automation Server" так же, как и над фреймворком "нижнего" можно охарактеризовать как огромный.

В ходе реализации верхнего уровня были отобраны десятки образцов микрокомпьютеров, производимые различными компаниями на мировом рынке. Исследовались платформы, которые потенциально могли бы выполнить роль – Hub, т.е. аппаратной платформы-носителя фреймворка "верхнего" уровня.

С каждым образцом проводилась работа по изучению его аппаратного состава вплоть до версии ARM ядра центрального процессора и набора инструкций который он поддерживает, сборки операционной среды и наполненности ее репозитория, коммуникационных возможностей, видов физических интерфейсов на плате, существующие на рынке корпуса и платы расширения для него, и это далеко не полный перечень мероприятий которые выполнялись с каждым микрокомпьютером.

После отбора очередного кандидата на нем разворачивалась вся системная инфраструктура Hub, а после устанавливался фреймворк и начиналось тестирование работы нашего программного комплекса на данной аппаратной платформе.

Мы испытывали образцы выпускаемые: китайскими производителями (Orange Pi, Banana Pi, Mango Pi, Radxa), европейскими (Raspberry Pi), американскими (Nvidia). В ходе тестов выяснилось, доступность китайских образцов самая высокая, при этом стоимость как правило ниже европейских и американских, но качество прошивок, системного софта, поддержки, наличие русскоговорящего комьюнити – не удовлетворительное.

Качество и аппаратного и программного обеспечения европейских и американских производителей высокое. Но есть проблемы с доступностью, из-за санкций полагаться на серийное производство на базе американских плат, в частности Nvidia у которых великолепный функционал и большой запас по производительности невозможно. Наилучшие результаты тестов были получены на Raspberry Pi версий RPi 5, RPi4, RPi CM 4.

При умеренных ценах на образцы с большим объемом оперативной и flash памяти мы получали очень хорошую производительность, продуманную сборку ОС Linux, наличие развитых вариантов в формате Compute Module, что очень важно для нашего проекта так как для сектора промышленной автоматизации нам требовалось найти подходящий вариант и модуля, и

корпуса Hub. В случае с Raspberry Pi все это доступно и развито. Также выявлено не высокое тепловыделение, даже при значительной нагрузке на Hub.

Таким образом в качестве аппаратной платформы Hub на текущий момент выбор аппаратной платформой для Hub пал на модули Raspberry Pi версии 4 и 5. Исполнение модулей – самодостаточная плата с распаянными разъемами (Raspberry Pi 5) и вариант с мезонинными модулями (Raspberry Pi Compute Module 4). С объемом оперативной памяти от 4 до 8 Гб и объемом eMMC flash памяти от 16 до 32 Гб.

На базе отобранных модулей в дальнейшем была проведена большая работа DevOps инженеров по кастомизации информационно-измерительных систем, баз данных, сервисных утилит, которые вместе с серверным фреймворком составляют программную платформу Hub.

Стараниями наших специалистов удалось не только скомпоновать серверные компоненты необходимые для работы платформы, но и найти, скомпилировать и установить web интерфейсы для большинства установленных серверных систем. Это делает наш комплекс чрезвычайно удобным как при разработке, так и при эксплуатации.

Пользователю не нужно искать "толстые" клиенты, и устанавливать их на свои рабочие ПК. Благодаря продуманному выбору используемых в проекте информационно-измерительным системам, большая часть из них уже имеет настроенный нами web интерфейс, и для подключения к ним достаточно web браузера. Для тех компонентов, которые "генетически" не имеют своего web интерфейса, например MQTT брокер Mosquitto, но при этом это очень важный и распространенный во всех типов проектов компонент, для таких компонентов мы постарались найти open source проекты клиентов с web интерфейсом, которые мы в ходе работ кастомизировали и установили на платформу Hub.

Результат работы непосредственно над программной реализацией фреймворка мы считаем весьма успешной. Здесь нам также удалось реализовать существенно больший функционал, чем тот который был изначально определен в ТЗ проекта.

В очередной раз мы убедились в правильности архитектурных решений, которые были приняты на самом начальном этапе проекта. При разработке служб серверного фреймворка мы использовали подходы, которые наработали при реализации фреймворка ПЛК, это было не приятными стечениями обстоятельств, а именно благодаря заранее продуманной архитектуре и техническими решениями стыковки двух фреймворков.

В результате продуманного подхода к реализации ПЛК, Hub и ПТК в целом, нам удалось получить на выходе уникальную технологию построения систем автоматизации на ее базе. В результате специально заложенных решений в архитектуру системы, наши компоненты можно использовать в довольно большом количестве комбинаций.

Вот первый вариант построения систем автоматизации только на ПЛК, это проекты, где компонент Hub отсутствует физически. В этом случае весь

прикладной код работает только на ПЛК. Причем в проекте может быть сколько угодно много географически распределенных ПЛК модулей. Которые при необходимости могут обмениваться данными своих измерительных каналов, или работать совершенно автономно.

Второй вариант построения систем автоматизации на базе наших компонентов. В этом варианте используются и ПЛК и Hub. Это самая мощная по функционалу система. Здесь в проекте по-прежнему, может быть, большое число ПЛК обслуживаемых одним Hub, ограничения здесь закладываются лицензионные, а не технические. В данном варианте возможно построение самых разнообразных систем с графическим интерфейсом на базе web технологии. Пользователю доступен оперативный сбор и реакция контроллера ПЛК и мощь серверной обработки Hub.

Пользователю доступно, фактически неограниченное количество дашбордов в системе потокового программирования – Node-Red, дашборды метрик в сервере логирования – Graylog, дашборды в мощной промышленной системе сбора метрик и алертов – Grafana.

Также в этом случае пользователю доступна вся мощь NoSQL базы данных MongoDB. Все собранные ПЛК данные будут сохраняться в БД. И нужно помнить, что в данном случае, сам Hub также выступает генератором измерительных каналов, которые никак не связаны с ПЛК, так как он может забирать данные большим числом способов из сторонних информационно-измерительных систем.

Третий вариант подразумевает, что источники первичной информации для разрабатываемой системы автоматизации уже достаточно интеллектуальные и сами могут тем или иным протоколом отдать данные. Типовым примером такой ситуации являются компоненты современных систем типа умный дом. Здесь датчики являются полностью самодостаточными элементами и Hub может общаться с ними по тому или иному протоколу непосредственно, без участия ПЛК. В таких проектах можно полностью обойтись без ПЛК, так на самом Hub есть механизмы создания измерительных каналов без физической основы непосредственно подключенных сенсоров (реализованная в нашем проекте так называемая концепция виртуальных устройств). В этом случае вся работа пользователя в части прикладного проекта будет выполняться непосредственно в Hub, а результат может передаваться на любые внешние системы если того требует реализация проекта.

Были показаны только три базовых варианта применения нашей платформы в проектах, связанных с автоматизацией, реализацией образовательных стендов и программ. Далее пользователю доступно множество комбинаций, состоящих из этих трех базовых вариантов.

Помимо технических достижений, команде проекта есть чем гордиться и в области продвижения продукта на рынок. На данный момент ООО "Горизонт" осуществило поставку комплектов оборудования для проведения лабораторных работ по физике на базе разработанных фреймворков, а также

оснащение класса для занятий по направлению кибер-физика специально подготовленного варианта нашей платформы – "Horizon Automation Cyber Physics" в ФГБОУ ВО «Самарский государственный технический университет». Реализация внедрения платформы ведется совместно с Домом Научной Коллаборации имени Н.Н. Семенова при Самарском Государственном Техническом Университете.

7. КРАТКИЕ ВЫВОДЫ ПО РЕЗУЛЬТАТАМ ВЫПОЛНЕНИЯ НИОКР

7.1 РАЗРАБОТАНА АРХИТЕКТУРА ПТК "HORIZON AUTOMATION"

Созданы два фреймворка:

1. "Horizon Automation PLC" для управления ПЛК на базе микроконтроллеров (ARM, Tensilica Xtensa) с использованием JavaScript (Espruino);
2. "Horizon Automation Server" для управления Hub на базе микрокомпьютеров (Raspberry Pi, x86/ARM) под управлением Linux;
3. Реализована микросервисная архитектура, обеспечивающая гибкость и масштабируемость.

7.2 КЛЮЧЕВЫЕ ДОСТИЖЕНИЯ

1. Успешно преодолены основные риски – потенциальная нехватка ОЗУ;
2. на ПЛК: оптимизация кода, тестирование на реальных устройствах (STM32, ESP32);
3. Ограничения сетевых соединений: реализация протокола LNP, интеграция MQTT и WebSocket;
4. Перехват управления REPL: создание службы RouteREPL для удаленного доступа;
5. Разработаны модули для взаимодействия с сенсорами, актуаторами, MQTT-брокерами, WebSocket-серверами;
6. Внедрена концепция цифровых двойников для сенсоров и актуаторов, которая позволила вывести работу пользователей с системами автоматизации на новый уровень.

7.3 ТЕСТИРОВАНИЕ И ВАЛИДАЦИЯ

1. ПЛК протестирован на платформах: ARM (STM32) и Tensilica Xtensa (ESP32);
2. Hub протестирован на платформах: Raspberry Pi 4/5 и Compute Module 4;
3. Подтверждена работоспособность: протокола LNP, интеграция с Grafana, Graylog, MongoDB, Node-Red.

7.4 ПРАКТИЧЕСКАЯ ЗНАЧИМОСТЬ

Платформа поддерживает три сценария использования:

1. Автономное применение ПЛК (промышленная автоматизация;
2. Комбинация ПЛК + Hub (промышленная автоматизация, сбор данных);
3. Работа только с Hub (виртуальные устройства, интеграция с IoT).

Реализована коммерческая поставка разработанного продукта и комплектов оборудования для образовательных программ в ФГБОУ ВО «Самарский государственный технический университет», а также внедрение в образовательные программы (Дом Научной Коллаборации при СамГТУ).

7.5 ДАЛЬНЕЙШИЕ НАПРАВЛЕНИЯ:

1. Адаптация фреймворка под архитектуру RISC-V.
2. Разработка форка Espruino для устранения ограничений интерпретатора.
3. Расширение списка поддерживаемых устройств и протоколов.

Итог: Проект успешно выполнен. Созданная платформа демонстрирует высокую гибкость, соответствие требованиям промышленной и образовательной автоматизации, а также потенциал для масштабирования.

8. ОЦЕНКА ПОЛНОТЫ ВЫПОЛНЕНИЯ ПОСТАВЛЕННЫХ ЗАДАЧ

Выполненные архитектурные задачи:

- Архитектура классов поддержки датчиков и актуаторов;
- Разработан модуль DeviceManager, обеспечивающий единое управление сенсорами и актуаторами, включая их регистрацию, сбор данных и управление.

Соответствие ТЗ: Полное.

Выполненные задачи, связанные с подсистемой REPL:

- ПО подсистемы перехвата среды REPL Espruino;
- Создан модуль RouteREPL, реализующий удаленный доступ к REPL через: любой из существующих COM портов ПЛК, USB порт, TCP соединение .

Соответствие ТЗ: Полное.

Выполненные задачи по разработке ПО обмена данными по протоколам WebSocket и MQTT. Разработаны модули:

- WSServer (WebSocket-сервер);
- MQTTGW (MQTT-шлюз) ;
- ПроxyWS/ПроxyMQTT (прокси-службы для интеграции с ядром фреймворка);
- Проведены тесты на ARM и TENSILICA XTENSA.

Соответствие ТЗ: Полное.

Выполненные задачи по разработке и отладке оригинального протокола обмена данными LHP:

- Протокол LHP, основанный на WebSocket и JSON, реализован и протестирован.

Соответствие ТЗ: Полное.

Выполненные задачи по разработке серверного ПО на базе Node-Red и разработка поддержки MongoDB:

- Интеграция с Node-Red выполнена через разработку системных узлов и потоков фреймворка в среде;
- Модуль ProviderMDB обеспечивает взаимодействие с MongoDB.

Соответствие ТЗ: Полное.

Выполненные задачи по стендовым испытаниям:

- Проведены множественные тесты на работу ПЛК (ESP32, STM32) в автономном режиме, а также тесты на работу во взаимодействии с Hub на базе Raspberry Pi.

Соответствие ТЗ: Полное.

Выполненные задачи по разработке аппаратного обеспечения:

- Аппаратное обеспечение реализовано на основе технических решений заводского изготовления ESP32, STM32F4 и Raspberry Pi;
- Прототипирование конструкции колесной платформы ("материнская плата", крепление модулей);
- Программная реализация сенсоров лазерный дальномер, датчики CO/CO₂, сервоприводы;
- Реализация платформы по заданным габаритам и весу (200x100x150 мм, до 0.5 кг).

Соответствие ТЗ: Полное.

Выполненные работы по поддержке работы датчиков и темпа считывания и передачи данных на серверную (Hub) платформу:

- Поддержка 128 датчиков/актуаторов – подтверждено тестами;
- Темп считывания данных ≥ 10 измерений/сек – подтверждено тестами;
- Темп передачи данных ≥ 4 измерений/сек – подтверждено тестами.

Соответствие ТЗ: Полное.

Патентная охрана:

- Реализованы мероприятия по защите патентного права. Получено свидетельство № 2025613291 от 27 января 2025г. о государственной программы для ЭВМ «Программный комплекс автоматизированного сбора, обработки, аналитики и визуализации данных от - ПЛК, датчиков, систем промышленной автоматизации, IoT систем, систем "умный дом", иных информационно-измерительных систем: "Horizon Automation Server".

Соответствие ТЗ: Полное.

Итоговая оценка полноты:

- Выполнено: 100% задач.

9. РЕКОМЕНДАЦИИ И ИСХОДНЫЕ ДАННЫЕ ДЛЯ ИСПОЛЬЗОВАНИЯ РЕЗУЛЬТАТОВ НИОКР

Разработанный программно-технический комплекс "Horizon Automation" и запатентованные фреймворки "Horizon Automation PLC" и "Horizon Automation Server" могут быть применены в следующих направлениях:

1. Промышленная автоматизация

Рекомендации:

- Внедрение в системы мониторинга за состоянием оборудования;
- Применение ПЛК с фреймворком "Horizon Automation PLC" для сбора данных телеметрии производственных линий, с темпом обновления данных не хуже 10 раз/сек;
- Применение ПЛК с фреймворком "Horizon Automation Server" для управления объектами производственных линий, с темпом обновления данных не хуже 10 раз/сек;
- Выполнение Hub-м роли центра обработки данных с дальнейшей выдачей готовых или промежуточных данных в SCADA, MES, ERP системы на базе поддерживаемых фреймворком "Horizon Automation Server" протоколов;
- Выполнение Hub-м роли центра обработки и визуализации данных технологических процессов на базе компонентов фреймворка "Horizon Automation Server" Node-Red и Grafana.

2. Образовательные программы и учебные стенды

Рекомендации:

- Создание лабораторий для проведение занятий по дисциплинам физика, информатика, робототехника; IoT и робототехники;
- Использовать ПЛК на базе ESP32 или STM32 для обучения студентов основам программирования микроконтроллеров на JavaScript;

Исходные данные:

- Учебные кейсы (например, управление LED-лентой, сбор данных с датчика влажности);
- Список совместимых сенсоров (DHT22, HC-SR04);
- Разработка курсов по промышленной автоматизации;
- Внедрить Hub на базе Raspberry Pi для демонстрации работы распределенных систем (связь ПЛК → Hub → облако).

Исходные данные:

- Методические материалы по настройке LHR-протокола.
- Примеры проектов (умная теплица, конвейерная линия);

3. Умный дом и IoT-решения

Рекомендации:

- Управление домашней автоматизацией:
- Подключить ПЛК к датчикам движения, освещения, климат-контролю, системе вентиляции и отопления, при этом Hub задействовать как центральный шлюз для интеграции и взаимодействия с голосовыми ассистентами (Яндекс Алиса, Amazon Alexa, Google Home).

Исходные данные:

- Схема размещения устройств в доме.
- Требования к безопасности (шифрование MQTT-трафика).
- Мониторинг энергопотребления:
- Настроить сбор данных с умных счетчиков через MQTT и их визуализацию в Grafana.

Исходные данные:

- Модели управляемых счетчиков и реле;
- График тарификации электроэнергии.

4. Сельское хозяйство и экологический мониторинг

Рекомендации:

- Автоматизация теплиц:
- Использовать ПЛК для ситуационного и прогнозного управления поливом, вентиляцией и освещением на основе данных с датчиков почвы (влажность, pH), окружающей среды (CO₂, температура) и прогнозных данных от гидрометцентра.

Исходные данные:

- Карта теплицы с зонами контроля;
- Алгоритмы управления (например, PID-регуляторы для температуры).
- Мониторинг качества воздуха:
- Развернуть сеть датчиков (PM2.5, CO) с передачей данных через Hub в облачное хранилище для анализа.

Исходные данные:

- Точки размещения датчиков;
- Требования к точности измерений (например, $\pm 5\%$ для CO).

5. Исходные данные для старта проекта

Рекомендации для успешного внедрения результатов НИОКР необходимы:

1. Аппаратная база:

- ПЛК: на базе микроконтроллеров STM32F4 или ESP32, под управлением фреймворка "Horizon Automation PLC";
- Hub: на базе микрокомпьютера Raspberry Pi 4/5, Orange Pi 5, под управлением фреймворка "Horizon Automation Server";
- Датчики/актуаторы: в соответствии со списком совместимых устройств.

2. Программное обеспечение:

- Установочные пакеты контроллерного фреймворка – "Horizon Automation PLC";
- Установочные пакеты фреймворка для Hub – "Horizon Automation Server", а также серверные службы платформы.

3. Документация:

- Конфигурационные шаблоны для network.json, device.json;
- Инструкции по настройке LHP протокола и взаимодействию с Hub.

Рекомендации по минимизации рисков:

- Аппаратная зависимость:
- Использовать модули с поддержкой PSRAM (например, ESP32-WROVER) для увеличения объема оперативной памяти.
- Производительность JavaScript:
- Оптимизировать код через минификацию и использование WebAssembly для критичных к скорости задач.
- Совместимость:
- Добавить поддержку промышленных протоколов (Modbus RTU, OPC UA) через плагины в Node-RED.

Пример успешного внедрения:

1. Проект: "Умная аудитория" для Самарского Государственного Технического Университета.

Задачи:

- Контроль освещения и климата в учебных классах.
- Интеграция с расписанием занятий через API.

Реализация:

- 20 ПЛК на базе ESP32 с датчиками движения и температуры.
- Центральный Hub на Raspberry Pi 5 с визуализацией в Grafana.

Эффект:

- Снижение энергопотребления на 30%.
- Автоматизация проветривания помещений.

2. Проект создания нового по формату и содержанию учебного курса по киберфизике на базе структуры "Дом научной коллаборации" в составе ФГБОУ ВО «Самарский государственный технический университет». На базе платформы сформирован учебный класс на 15 мест, полностью подготовленный для проведения практических и лабораторных занятий по курсу кибер-физика.

ИТОГ: Программно-аппаратная платформа, полученная в результате НИОКР позволяет создавать гибкие и экономичные решения для автоматизации в промышленности, образовании и IoT. Для старта проекта достаточно использовать доступные аппаратные компоненты и открытое ПО, при этом низкий уровень входа (объема технических знаний) делает платформу исключительно доступной для применения в домашней автоматизации (DIY).

Для применения в диспетчеризации и управлении технологическими процессами промышленных предприятий платформа предоставляет богатые возможности по взаимодействию с всевозможными датчиками и исполнительными устройствами, а также спектр поддерживаемых промышленных протоколов и мощные вычислительные возможности Hub-а платформы.

Для учебных заведений перспективным выглядит применение платформы для быстрой организации образовательных программ и курсов по направлениям: кибер-физика, информатика, робототехника, организация разработки и прототипирования БПЛА, кибер-химия. Уровень учебных заведений, на базе которых возможна организации подобных курсов – от общеобразовательных школ и кванториумов до университетов и исследовательских центров.

10. ОЦЕНКА ЭКОНОМИЧЕСКОЙ ЭФФЕКТИВНОСТИ

1. Снижение затрат:

- Использование open-source технологий (Espruino, Node.js, MongoDB, Node-Red) сокращает лицензионные расходы;
- Унификация архитектуры и повторное использование модулей уменьшают затраты на разработку новых проектов.

2. Оптимизация эксплуатации:

- Централизованное управление через Hub снижает затраты на обслуживание распределенных систем;
- Автоматизация сбора данных и обработки ошибок минимизирует необходимость ручного вмешательства.

3. Повышение конкурентоспособности:

- Поддержка образовательных программ (на примере поставки комплектов оборудования для образовательных программ в ФГБОУ ВО «Самарский государственный технический университет» и реализация проекта с Домом Научной Коллаборации) открывает новые рынки;
- Возможность интеграции с IoT, промышленной автоматизацией и системами "Умный дом" расширяет целевую аудиторию.

4. Риски:

- Зависимость от аппаратных платформ (например, Raspberry Pi) может привести к дополнительным затратам при смене поставщиков.
- Необходимость обучения персонала работе с новыми инструментами (JavaScript, Node-Red) требует временных и финансовых вложений.

Система "Horizon Automation" демонстрирует высокий потенциал для внедрения в проекты автоматизации благодаря:

- Технической универсальности и надежности и зрелости технологий;
- Экономии за счет open-source решений и высокой "плотности" технологий и функциональных возможностей ПЛК и Hub;
- Широкие возможности масштабирования под задачи любой сложности.

Ключевые преимущества:

- Для промышленности: «быстрая адаптация» ПЛК и Hub под спектр задач предприятия, поддержка распределенных систем, мониторинг и уведомления в реальном времени.
- Для образования: наработки для организации курсов по кибер-физике, робототехнике, информатике, прототипированию БПЛА, IoT, кибер-химии.
- Для малого бизнеса: многофункциональность при небольших затратах за счет продуманных технических и аппаратных решений.

11. ОЦЕНКА НАУЧНО-ТЕХНИЧЕСКОГО УРОВНЯ ВЫПОЛНЕННОЙ НИОК В СРАВНЕНИИ С ЛУЧШИМИ ДОСТИЖЕНИЯМИ

1. Инновационность и уникальность решений

➤ Создание протокола LHP:

Разработанный протокол LHP, основанный на WebSocket и JSON, позволяет унифицировать взаимодействие как между внутренними компонентами платформы так и внешними информационно-измерительными системами. В отличие от стандартных протоколов (OPC, ModBus, KNX), LHP является простым, универсальным, быстрым как в работе так и в освоении протоколом. Позволяет просто и логично организовать двухсторонний обмен данными между системами.

Сравнение с аналогами: Промышленные системы чаще используют группу OPC протоколов, которая является "тяжелой", вычислительно затратной технологией. На основе LHP легко организовать обмен между большим числом "простых" IoT устройств;

➤ Использование JavaScript для программирования ПЛК:

Применение Espruino (JavaScript-интерпретатор) на микроконтроллерах — новаторский подход. Традиционно ПЛК программируются на языках стандарта IEC 61131-3 который определяет такие языки как LD (Ladder Diagram), FBD (Function Block Diagram), SFC (Sequential Function Chart), ST (Structured Text) и IL (Instruction List).

Указанные языки являются исключительно узкоспециализированными, при этом они много лет не обновлялись, а создавались во времена когда не существовала рынка недорогих высокопроизводительных микроконтроллеров аналогичные АРМ STM32F40x Cortex M4, ESP32, RISC-V, также на момент разработки данных языков не существовало компактных высокопроизводительных промышленных одноплатных микрокомпьютеров, аналогичных Raspberry Pi и др. Подход данных языков подразумевал сильную специализацию специалистов при выполнении проектов автоматизации. Одни группы занимались только полевым уровнем, другие программированием на одном из языков IEC 61131-3 ПЛК контроллеров, другие работали только с серверными компонентами проекта, где были совершенно другие технологии (OPC) и языки программирования.

Такой подход требует целый штат узкоспециализированных специалистов, что в наше время – синергии является нерентабельным а многим компаниям совершенно не достижимым в том числе в связи с кадровыми проблемами которые имеют над национальный характер.

Решением является разработка технологий, с которыми могут работать уже существующие на рынке труда в большом количестве специалисты. А это значит нужны технологии автоматизации, построенные на языках общего назначения. JavaScript является представителем таких языков и имеет на земле самое большое комьюнити.

Сравнение с аналогами: Решения от Siemens, Rockwell Automation и других "китов" производителей ПЛК сохраняют фокус на классических ПЛК языках, тогда как подход платформы "Horizon Automation" меняет порог входа для разработчиков не знакомых с JS, а также позволяет набрать и до обучить специалистов из существующего большого комьюнити.

➤ Цифровые двойники сенсоров/актуаторов:

Реализация концепции цифровых двойников на уровне как ПЛК, так и Hub позволяет абстрагироваться от аппаратных особенностей устройств.

Сравнение с аналогами: Аналогичная функциональность есть в платформах PTC ThingWorx и Siemens MindSphere, но их внедрение требует значительных ресурсов и являются дорогостоящими.

2. Технологическая продвинутость

➤ Микросервисная архитектура:

Разделение системы на независимые модули (Network, WSServer, MQTTGW и многие др.) повышает гибкость и масштабируемость.

Сравнение с аналогами: Промышленные IoT-платформы (AWS IoT, Microsoft Azure IoT) также используют микросервисы, но их реализация на уровне ПЛК практически не встречается.

➤ Интеграция современных инструментов:

Использование Grafana для визуализации, MongoDB для хранения данных и Node-RED для потокового программирования соответствует трендам Industry 4.0.

Сравнение с аналогами: Аналогичные стеки применяются в Bosch IoT Suite и IBM Watson IoT, однако "Horizon Automation" ориентирован на малые и средние проекты.

➤ Поддержка мультиплатформенности:

Работа на ARM, TENSILICA XTENSA и RISC-V обеспечивает гибкость выбора аппаратной базы.

Сравнение с аналогами: Промышленные контроллеры (например, от Beckhoff) часто привязаны к конкретным архитектурам.

3. Слабые места в сравнении с лидерами отрасли

- Производительность JavaScript на ПЛК:
- Исполнение кода на интерпретируемом языке может уступать по скорости решениям на C/C++ или FPGA-платформам.
- Проприетарный протокол LHP:
- Отсутствие поддержки LHP сторонними системами усложняет интеграцию с существующей инфраструктурой предприятий.

3. Позиционирование на фоне мировых аналогов

Критерий	Horizon Automation	Промышленные решения (Siemens, Rockwell)	Образовательные платформы (Arduino, Raspberry Pi)
Целевая аудитория	Промышленные предприятия, средний и малый бизнес, сфера образования, IoT-стартапы	Крупная промышленность	Образование, DIY-проекты
Гибкость архитектуры	Высокая (микросервисы, мультиплатформенность)	Низкая	Высокая
Стоимость внедрения	Низкая (open-source компоненты)	Высокая	Низкая
Надежность	Средняя	Высокая	Низкая (не для промышленного использования)

Итоговая оценка научно-технического уровня

Проект "Horizon Automation" демонстрирует высокий уровень инноваций в области образовательной и маломасштабной автоматизации, сочетая современные подходы (микросервисы, цифровые двойники) с доступностью. Однако в сравнении с промышленными решениями уступает в:

- Надежности и наработке на отказ. Производительности из-за использования интерпретируемых языков и массовых аппаратных платформ;
- Стандартизации – так как использует богатый набор протоколов и стандартов, что усложняет стандартизацию;

Сильные стороны:

- Высокая универсальность и многофункциональность при низкой стоимости;
- Низкий порог овладения навыками разработки на базе платформы;
- Уникальные возможности для организации образовательных программ.